

Abstract. Deep Reinforcement Learning (DRL) has evolved from a research curiosity capable of mastering Atari games into a foundational ingredient of today's most capable AI systems — from humanoid robots fluently manipulating household objects, to LLM-powered agents that browse the web, write code, and conduct multi-step scientific reasoning. This report surveys the DRL landscape across three axes: (1) the algorithmic foundations from DQN to Decision Transformer; (2) the simulation platforms and open-source ecosystem that enable modern DRL research; and (3) the rise of *agentic AI* — large-model agents that combine RL fine-tuning (RLHF/RLAIF/RLVR) with tool use, planning, and embodiment. We analyze representative SOTA systems from 2024–2026 across robotics, game AI, and AI for science, identify five cross-cutting research trends (world models, diffusion policies, foundation agents, sim2real, RL+Transformers), and provide a comparative analysis of five algorithm families along axes of sample efficiency, stability, and real-world deployability. A small reproducible Bonus MVP — a PPO agent on classic control plus a tool-using LLM agent — is included to make the survey concrete.

Keywords: Deep Reinforcement Learning, Agentic AI, RLHF, RLAIF, Vision-Language-Action Models, Diffusion Policy, World Models, Foundation Agents, Sim2Real.

Part 1 — Introduction to Deep Reinforcement Learning

1.1 Fundamental Concepts

1.1.1 Reinforcement Learning (RL)

Reinforcement Learning（強化學習）研究的是一個 **agent** 如何在與 **environment** 反覆互動的過程中，學會選擇能最大化長期累積回饋（cumulative reward）的行動。RL 與監督式 / 非監督式學習的本質差異在於：訓練訊號不是事先標好的標籤，而是由 **agent** 自己用行動「探索」出來的回饋。這個 closed-loop 的結構讓 RL 同時面對三個傳統 ML 不必處理的難題——延遲回饋（credit assignment）、非穩態資料分布（policy 改變則資料分布跟著改變）、以及 **探索 vs 利用** 的權衡。

1.1.2 Markov Decision Process (MDP)

絕大多數 DRL 演算法都假設環境可以被建模成一個 **Markov Decision Process** $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ ：

- $\mathcal{S}$ ：狀態空間。
- $\mathcal{A}$ ：動作空間（離散或連續）。
- $P(s' \mid s, a)$ ：轉移機率，滿足 **Markov property**——下一狀態只依賴當前 (s, a) ，不依賴歷史。

- $R(s, a)$ 或 $R(s, a, s')$ ：reward function。
- $\gamma \in [0, 1]$ ：discount factor，把無窮時域問題折成有限期望值。

當部份可觀測時退化成 **POMDP**（Partially Observable MDP），agent 只看得到 observation o_t 而非 s_t ，這是真實機器人和大多數 video games 的設定。

1.1.3 Reward Function

Reward function 是 RL 中最重要、也最容易被低估的設計選擇。它決定「目標是什麼」，因此一切學到的行為都是 reward 的副產物——reward hacking、specification gaming 都源自此處。SOTA 趨勢之一是不再人工手刻 **reward**，而是用 LLM 生成（如 NVIDIA Eureka）、用人類偏好學（RLHF）、或從可驗證的成功訊號取得（RLVR，如 DeepSeek-R1）。

1.1.4 Policy 與 Value Functions

- **Policy** $\pi(a \mid s)$ ：在每個狀態下選擇動作的分布。Deterministic 時寫成 $\pi(s) \rightarrow a$ 。
- **State-value** $V^\pi(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s]$ ：從 s 起遵循 π 的期望累積折扣回饋。
- **Action-value** $Q^\pi(s, a)$ ：在 s 先執行 a 再遵循 π 的期望回饋。
- **Advantage** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ ：執行 a 比平均策略好多少，是 policy gradient 方法的核心訊號。

兩條基礎理論：**Bellman** 方程將 V 、 Q 寫成遞迴形式，為所有 value-based 演算法提供更新公式；**Policy Gradient Theorem** 給出 $\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a \mid s) Q^{\pi_{\theta}}(s, a)]$ ，是 PPO/SAC 等 policy-based 方法的根。

1.1.5 Exploration vs Exploitation

短期想最大化 reward 就應該選目前看起來最好的動作（exploit），但長期想找到真正的最優策略就必須嘗試陌生動作（explore）。常見策略：

策略	機制	典型用法
ϵ -greedy	以機率 ϵ 隨機、否則 greedy	DQN 系列
Boltzmann/softmax	依 Q 值加溫度做機率取樣	tabular RL
Entropy bonus	在 loss 加入策略熵 $\mathcal{H}(\pi)$	A2C / PPO / SAC
Intrinsic motivation	額外的 curiosity / prediction-error reward	RND, ICM
Posterior sampling	從後驗分布取樣 Q	Bootstrapped DQN
Noise on parameters	NoisyNet	Rainbow DQN

1.2 Deep RL Algorithms — Survey and Comparison

下面對 10 個代表性演算法做系統性整理，每個都包含 **core idea** / **advantages** / **weaknesses** / **typical applications**。

1.2.1 DQN (Deep Q-Network)

Core idea. Mnih et al., Nature 2015 ¹。用神經網路 $Q_{\theta}(s,a)$ 近似 action-value，搭配兩個關鍵 trick 解決深度 RL 的不穩定：1. **Experience Replay Buffer**：把 (s,a,r,s') 存進 buffer，訓練時均勻取樣，打破時序相關性。2. **Target Network**：另外維護一個延遲更新的 Q_{θ^-} ，提供穩定的 target $y = r + \gamma \max_a Q_{\theta^-}(s', a)$ 。

Advantages. 第一個在 high-dimensional 像素輸入上成功的 DRL 演算法；架構簡單；off-policy 因此可重複利用資料。

Weaknesses. $\max$ 運算造成系統性 Q 值高估（overestimation bias）；只能處理 離散動作；對 reward scaling 敏感；訓練不穩。

Typical applications. Atari 2600 benchmark、桌面遊戲、離散決策（推薦、廣告 bidding）。

1.2.2 Double DQN

Core idea. van Hasselt et al., AAAI 2016 ²。把目標的 $\max$ 拆成「選動作」與「估值」兩步： $y = r + \gamma Q_{\theta^-}(s', \arg \max_a Q_{\theta}(s', a))$ 。online network 負責挑動作、target network 負責估該動作的 Q，緩解高估偏差。

Advantages. 改動極小（一行 code），效果卻顯著穩定；後續所有 DQN 變體幾乎都內建。

Weaknesses. 仍是離散動作；overestimation 沒有完全消除（只是降低）；在動作空間很大時偏差可能轉為低估。

Typical applications. 所有原本 DQN 的場景；現代 Atari 訓練的預設 baseline。

1.2.3 PPO (Proximal Policy Optimization)

Core idea. Schulman et al., OpenAI 2017 ³。沿用 TRPO 的 trust-region 哲學（避免每次更新走太遠導致 policy 崩壞），但用一個簡單的 **clipped surrogate objective** 取代二階近似：
$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$
 其中 $r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ 。

Advantages. 目前最常用的 on-policy 演算法；實作簡單、超參數魯棒、適用離散與連續動作；是 RLHF / RLAIFF / RLVR 的事實標準（GPT、Claude、Gemini 全部用 PPO 變體做對齊）。

Weaknesses. on-policy 因此 sample-inefficient；對 advantage normalization、learning-rate schedule 等實作細節敏感（被 Andrychowicz et al., ICLR 2021 ⁴ 證實是「37 implementation details」）。

Typical applications. 機器人控制、自駕、Dota 2/StarCraft 等大規模分散式訓練、LLM 對齊（RLHF）。

1.2.4 A2C / A3C (Asynchronous Advantage Actor-Critic)

Core idea. Mnih et al., ICML 2016⁵。同時學一個 actor π_{θ} 和一個 critic V_{ϕ} ，用 advantage $\hat{A} = R - V_{\phi}$ 替代 Q 來降低 policy gradient 的 variance。**A3C** 用多個 worker 在 CPU 上非同步地各自抓資料、各自做梯度、push 回 master；**A2C** 是同步版本（等所有 worker 收完一筆 batch 再一起更新），在 GPU 時代往往效果更好。

Advantages. 不必 replay buffer；天然支援連續動作；多 worker 也是一種探索的隱性 randomization。

Weaknesses. 比 PPO 不穩；缺乏 trust-region 控制；on-policy 因此 sample-inefficient。已被 **PPO** 取代成 baseline。

Typical applications. 學術 baseline；多核 CPU cluster 上的大規模訓練（PPO 出現前的 Dota 等）。

1.2.5 SAC (Soft Actor-Critic)

Core idea. Haarnoja et al., ICML 2018⁶。把標準 RL 目標改成 **maximum entropy** 形式： $J(\pi) = \sum_t \mathbb{E}[\left[r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))\right]]$ 。 α 鼓勵 policy 在所有「等好」的動作上保持熵，自然兼顧探索。架構是 off-policy actor-critic + 兩個 Q network（取 min 避免高估）。

Advantages. 連續控制的 **SOTA baseline**（與 TD3 並列）；sample-efficient（off-policy）；temperature α 可自動調整；穩健。

Weaknesses. 實作比 PPO 複雜；連續動作專用（離散版本後出但較少用）；對 critic 過擬合敏感（需要 target 與 ensemble）。

Typical applications. 機械手臂、足式機器人、UAV、自駕。

1.2.6 TD3 (Twin Delayed DDPG)

Core idea. Fujimoto et al., ICML 2018⁷。針對 DDPG 的 Q 值高估與不穩，提出三個技巧：1. **Clipped Double Q-learning**：兩個 critic 取 min。2. **Delayed policy updates**：critic 多更新幾步 actor 才更新一次。3. **Target policy smoothing**：在 target action 加 Gaussian noise 後 clip，避免 critic 對窄峰 over-fit。

Advantages. Deterministic policy 在精確控制（如手臂末端定位）上略優於 SAC；演算法純粹、無 entropy term。

Weaknesses. 缺乏 SAC 的自動探索能力，對 exploration noise schedule 敏感；當任務需要多模態行為時，deterministic policy 表現不如 SAC。

Typical applications. 連續控制；機器人精準定位；常與 SAC 並列為 baseline。

1.2.7 MuZero

Core idea. Schrittwieser et al., Nature 2020⁸。AlphaZero 的繼承者，但不再需要環境模型作為已知輸入——MuZero 學一個抽象的 **latent dynamics model**（不直接預測 pixels，只預測對 policy/value 有用的 latent 表徵），然後在這個學到的模型裡做 MCTS。可以同時打敗 AlphaZero（圍棋/西洋棋/將棋）與 R2D2（Atari），不需要任何遊戲規則。

Advantages. model-based 帶來高 sample efficiency；不需手刻環境模型；同一架構征服 board games + Atari。

Weaknesses. 工程複雜（MCTS + 表徵學習 + reward/value prediction）；對隨機環境（如撲克）不擅長；訓練成本高。

Typical applications. 棋類遊戲、Atari、video compression（DeepMind 後續 paper）、推薦系統（少數產業應用）。

1.2.8 Decision Transformer (DT)

Core idea. Chen et al., NeurIPS 2021⁹。把 RL 重新當成序列建模問題——給定一個 trajectory $(R_1, s_1, a_1, R_2, s_2, a_2, \dots)$ ，用 Transformer 自迴歸地預測下一個動作。 R_t 是 **return-to-go**（從 t 起的剩餘 reward 總和）；inference 時把目標 return 餵進去，agent 就「假裝自己能拿到那麼多 reward」並輸出動作。

Advantages. 完全不用 RL（沒有 TD、沒有 policy gradient）；天然支援 offline RL；可以受惠於 Transformer scaling laws；用 return-conditioning 做 multi-task 很自然。

Weaknesses. 在 **stitching**（從不連續的次優 trajectory 拼出最優策略）上不如 dynamic programming-based offline RL（如 CQL）；對 OOD return target 敏感；需要大量資料才打得贏 TD-based 方法。

Typical applications. Offline RL benchmarks、robotic learning from demonstrations、後續 Trajectory Transformer / Gato / VLA 系列的祖先。

1.2.9 Offline RL

Core idea. 給定一個靜態的 dataset $\mathcal{D} = \{(s, a, r, s')\}$ （由未知 behavior policy 收集），不能再與環境互動，學一個 policy 使其表現超過 behavior。核心難題是 **distribution shift**：學到的 policy 在 OOD 動作上的 Q 估計沒有資料約束，會「幻想」出很高的值。

代表方法。- **BCQ** (Fujimoto et al., ICML 2019¹⁰)：強制 policy 只選與 behavior 相近的動作。- **CQL** (Kumar et al., NeurIPS 2020¹¹)：在 critic loss 加 conservative penalty，明確壓低 OOD action 的 Q。- **IQL** (Kostrikov et al., ICLR 2022¹²)：用 expectile regression 學 V ，再做 advantage-weighted regression 取 policy，完全避開對 OOD 動作的 query。- **Decision Transformer / Trajectory Transformer**：序列建模路線。

Advantages. 不需要環境互動，適合 昂貴/不安全/不可逆 的場景（醫療、金融、自駕真實道路）。

Weaknesses. 上限受限於 behavior policy 覆蓋範圍；對 OOD 的處理仍是 open problem；缺少 well-established benchmark 之外的真實成功案例。

Typical applications. Medical AI（ICU 治療策略）、推薦系統、自駕（從 fleet 紀錄學）、機器人從 demonstration 學。

1.2.10 Hierarchical RL (HRL)

Core idea. 把長時域決策拆成兩層：**high-level policy** 選擇 sub-goal 或 option（持續多個 timestep 的動作）；**low-level policy** 在 option 內執行 primitive actions。經典框架是 **Options framework** (Sutton, Precup, Singh, 1999¹³)；現代代表如 **FuN** (FeUdal Networks, Vezhnevets et al., ICML 2017¹⁴) 和 **HIRO** (Nachum et al., NeurIPS 2018¹⁵)。

Advantages. 緩解 sparse reward 與 long-horizon credit assignment；sub-policy 可重用；天然有可解釋性（可以看到 sub-goal）。

Weaknesses. 怎麼自動發現 option 仍是 open problem；high-level 與 low-level 的訓練常彼此干擾；在純 end-to-end 大模型時代（如 OpenVLA、 π^0 ）反而被 flat policy 打過。

Typical applications. Long-horizon robotics、multi-room navigation、Minecraft 類的開放世界任務。

Part 2 — Survey of DRL Systems and Platforms

DRL 的進步從來不是只有演算法。一個能在 單一 GPU 上每秒模擬數萬個機器人的 simulator、一個能在 百萬時間步 內收斂的 distributed trainer，往往才是論文背後真正的決勝點。本章節整理十個現代 DRL 系統，沿四個維度做比較：系統架構 / 模擬環境 / 支援演算法 / 目標應用，並指出每個系統的優勢與限制。

2.1 NVIDIA Isaac Gym / Isaac Sim / Isaac Lab

架構。 Isaac Gym（2021 釋出）是 **GPU-end-to-end** 的機器人模擬器——physics simulation、observation rendering 與 RL training 全部在 GPU 上，不必走 CPU↔GPU 來回。一張 RTX 4090 可同時模擬數千到上萬個並行環境（envs），把 PPO 從「兩週」壓到「數小時」收斂。2023 年起 NVIDIA 推出 **Isaac Sim**（基於 Omniverse 的 photorealistic simulator）與 **Isaac Lab**（取代舊版 Isaac Gym 的學術用 RL 框架）；2024 年又跟 Google DeepMind、Disney 聯手發布 **Newton**——一個專為 robotics 設計的下一代 differentiable physics engine（建立在 NVIDIA Warp 上）。

支援演算法。 PPO、SAC、TRPO（透過 rl_games / skrl / Stable-Baselines3 整合）；2024–2025 起 NVIDIA 大力推 **Eureka / DrEureka**（用 LLM 自動生成 reward function）與 **GR00T**（humanoid foundation model）。

應用。 四足機器人（ANYmal / Spot）、人形機器人（GR00T training, 1X NEO）、機械手（Allegro / Shadow Hand）、Sim2Real 的所有環節。

優勢。 速度無人能敵；photorealistic rendering；NVIDIA 的 GPU/CUDA 生態。**限制。** 對非 NVIDIA GPU 不友善；deformable / fluid 支援仍弱；學習曲線陡（Omniverse 的 USD 格式並不友善）。

2.2 Habitat Lab (Meta AI)

架構。Meta AI 開發的 **embodied AI** 模擬平台。Habitat-Sim 用 C++ + Bullet/Magnum 做 high-throughput rendering (10000+ FPS in headless mode)，Habitat-Lab 是 Python 端的 RL/IL framework。2023 年釋出 **Habitat 3.0** 支援 humanoid avatar 和 human-robot interaction；2024 年 **Habitat-Matterport 3D Semantics (HM3D)** 提供 1000+ 棟真實住宅的 photorealistic 3D 掃描。

支援演算法。PPO、DD-PPO (distributed PPO)、imitation learning。

應用。Visual navigation (PointNav, ObjectNav, ImageNav)、language-guided navigation、mobile manipulation、social navigation。

優勢。室內具身導航的事實標準；rendering 與物理速度極快；資料集生態完整 (MP3D、HM3D、Gibson、ReplicaCAD)。**限制**。物理模擬不像 Isaac 那麼精準 (足以做 navigation 但 manipulation 較弱)；以 Linux 為主。

2.3 CARLA

架構。開源的都市自駕模擬器 (Intel + CVC + Toyota Research)，基於 Unreal Engine 4/5。提供完整的城市環境 (CARLA Towns)、感測器 (RGB、Depth、Lidar、Radar、IMU、GNSS)、可調節天氣、行人/車輛 NPC，以及 Python/C++ API。CARLA Leaderboard 是業界公認的端到端自駕 benchmark。

支援演算法。任意 DRL 演算法 (環境用 Gym/Gymnasium 包好即可)；近年大量出現基於 model-based RL (如 Latent World Models) 與 imitation learning + RL hybrid 的工作。

應用。自駕策略 / planning / control；多 agent driving；強化學習與規則式 planner 的對比。

優勢。唯一被產業界廣泛採用的開源自駕模擬器；社群活躍；CARLA Challenge 比賽提供共同 leaderboard。**限制**。Unreal 端 rendering 很重，難以同時跑大量 envs；sim2real gap 仍然大；CARLA scenarios 的 distribution 與真實道路不完全重疊。

2.4 Unity ML-Agents

架構。Unity 推出的開源 toolkit，讓任何 Unity 場景變成 Gym-compatible RL environment。架構分三層：**Unity scene + ML-Agents C# Behavior** → **Python low-level API (gRPC)** → **mlagents-learn trainer** (內建 PPO、SAC、POCA for multi-agent)。

應用。遊戲 AI 訓練 (NPC 行為、難度調節)、教育示範、small-scale robotics (藉由 Unity Robotics Hub 整合 ROS)。

優勢。對遊戲開發者最友善；3D 物件 / 動畫資產庫巨大；跨平台部署。**限制**。物理模擬不及 Isaac/MuJoCo；trainer 不及 RLlib 強大；不適合大規模 distributed training。

2.5 Ray RLlib

架構。UC Berkeley RISELab → Anyscale 商業化。建構在 **Ray** distributed computing 上，提供 **rollout workers / learner workers / replay buffer** 抽象，能輕鬆從單機擴到上千 CPU/GPU。內建超過 30 種演算法的官方實作（PPO, IMPALA, APPO, SAC, DQN, DDPG, MARL 等）。

應用。產業界的大規模 DRL 訓練；推薦系統、game AI、廣告 bidding；Anyscale 是目前最完整的 commercial DRL platform。

優勢。**scalability** 一流；演算法選擇齊全；產業級維護。**限制**。抽象多、API 學習曲線陡；2.x → 3.x 大改動讓社群一度抱怨穩定性；單機快速實驗反而不如 SB3 / CleanRL 順手。

2.6 Stable-Baselines3 (SB3)

架構。PyTorch 重寫的 Stable-Baselines（OpenAI Baselines 的 fork）。設計哲學是 **single-machine**、可讀性優先、**reproducibility** 優先。提供 PPO、A2C、DQN、SAC、TD3、DDPG 等核心演算法，所有 baseline benchmark 與 hyperparameter 公開在 **rl-baselines3-zoo**。

應用。研究 baseline、教學、小型產業 prototyping。

優勢。學術界最被引用的 **PyTorch DRL** 庫；文件完整；rl-zoo 提供了 100+ 個調好的 hyperparameter set。**限制**。不支援 distributed training（單 GPU + 多 env 上限）；演算法保守（不追新 SOTA 如 DreamerV3）。

2.7 FinRL (AI4Finance Foundation)

架構。哥倫比亞大學 Xiao-Yang Liu 團隊主導的開源金融 DRL 框架。三層架構：**FinRL-Meta**（金融資料 + 環境）→ **FinRL**（演算法層，基於 SB3 / ElegantRL / RLlib）→ **FinRL-Tutorials**（範例）。2024 年釋出 **FinRL-DeepSeek** 結合 LLM signal 與 DRL trader。

應用。股票 / 加密貨幣 / 期貨的演算法交易、portfolio optimization、market making、order execution。

優勢。唯一被廣泛採用的金融專用 **DRL** 框架；提供 trading environments、tutorials、leaderboard；活躍維護。**限制**。真實金融市場的 non-stationarity 讓 sim-trained policy 難以泛化；論文中的 outperformance 在實盤常打折扣。

2.8 MineDojo (NVIDIA + 學術界)

架構。Fan et al., NeurIPS 2022¹⁶。在 Minecraft 之上建造的 open-ended embodied agent benchmark。三大資產：1. **Simulator**（基於 MineRL，提供 thousands of 任務）；2. **資料集**：730k YouTube 影片 + 7000+ Wiki 頁 + 340k Reddit posts；3. **基礎模型 MineCLIP**：對齊 Minecraft 影片與文字描述。

應用。Voyager (Wang et al., NeurIPS 2023¹⁷) 在此上面驗證 LLM-driven lifelong learning；後續 GITM、JARVIS-1 等基於 MineDojo 做評估。

優勢。目前唯一規模化的開放世界 **embodied benchmark**；任務多樣性 + 海量 unlabeled video；極適合驗證 foundation agents。限制。Minecraft 本身的視覺/物理與真實世界差距大；setup 複雜（需要 Java + Mod 環境）。

2.9 OpenVLA (Stanford + Google DeepMind + Toyota Research, 2024)

架構。Kim et al., 2024¹⁸。開源 **Vision-Language-Action foundation model**，7B 參數，在 Open X-Embodiment 970k+ robot trajectory 上微調 Llama 2 + DINOv2/SigLIP visual encoder。輸出是 tokenized 7-DoF action sequence。

支援演算法。Primarily imitation learning + behavior cloning；可用 RL fine-tune（後續工作如 RLDP、PARL）。

應用。Multi-embodiment manipulation; benchmark 在 BridgeData-V2、LIBERO 等。

優勢。首個 **open-weights VLA**，打破 RT-2 的閉源壟斷；參數量適中（消費級 GPU 可微調）。限制。在新 embodiment 上仍需 demonstration fine-tuning；推論 latency 對 high-frequency control 是挑戰；缺乏線上 RL 機制。

2.10 Microsoft AirSim / Project AirSim

架構。Microsoft Research 的開源無人機/車輛模擬器，建構在 Unreal Engine 上。提供 photorealistic environments、physics-based flight dynamics 與 multiple sensor modalities。注意：原 AirSim repo 已 archived（2022 年底），Microsoft 把後續開發轉到商業化的 **Project AirSim**（基於 Azure cloud）。社群 fork **Cosys-AirSim** 繼續維護開源版。

應用。UAV 自主飛行訓練、深度視覺 + RL 控制、CV pipeline 測試。

優勢。學術界 **UAV RL** 的主要實驗平台；豐富感測器；Multi-vehicle 支援。限制。已停止官方開源維護是隱患；對沒用過 Unreal 的人 setup 不容易；rendering throughput 限制大規模並行訓練。

2.11 比較表

$p\{3.6\text{cm}\}p\{3.2\text{cm}\}p\{3.6\text{cm}\}$

平台	**主要 simulator / 環境**	**支援演算法**	**目標應用**
Isaac Lab / Sim	GPU end-to-end physics + photorealistic	PPO/SAC + Eureka, GR00T	Humanoid / 機械手 / quadruped
Habitat 3.0	Photoreal 室內 3D 掃描 (HM3D)	PPO / DD-PPO / IL	室內具身導航 + HRI
CARLA	Unreal-based 城市駕駛	任意 DRL (Gym API) + IL	自駕、planning
Unity ML-Agents	任意 Unity 場景	PPO / SAC / POCA	遊戲 AI、教學
RLlib	任意 Gym 環境 (Ray cluster)	30+ 演算法 (含 MARL)	大規模產業 DRL
Stable-Baselines3	任意 Gym 環境 (單機)	PPO / SAC / TD3 / DQN ...	研究 baseline、教學
FinRL	金融資料 (股票/加密)	透過 SB3/RLlib/ElegantRL	演算法交易、portfolio
MineDojo	Minecraft + 多模態資料	任意 + LLM agent	開放世界 / 終身學習
OpenVLA	Open X-Embodiment 資料集	IL (可 RL fine-tune)	多 embodiment manipulation
AirSim	Unreal 無人機 / 車輛	任意 DRL (Gym API)	UAV、autonomous driving

Part 3 — Agentic AI and Autonomous Agents

DRL 在 2024–2026 最大的「身分轉換」是：它從一個獨立的 **sub-field**，變成了大型模型（LLM/VLM）系統裡的一個 **alignment + reasoning** 元件。本章梳理 LLM agents、RLHF / RLAIIF、tool-using、multi-agent、self-improving、memory 等六個面向，最後討論 DRL 如何被融入 agentic AI、以及對 AGI 的影響。

3.1 LLM Agents — 三大產品堆疊

3.1.1 OpenAI Agent Stack

- **Computer-Using Agent (CUA) / Operator** (2025 Jan)：GPT-4o vision 模型 + RL 訓練的 CUA，能在瀏覽器中點擊、輸入、滾動；2025 升級到 o3-based 版本，加入 self-correction loop 與遇到不確定時主動 hand-back 給用戶。
- **Deep Research** (2025 Feb)：以 o3 為核心的多步研究 agent，能瀏覽網頁、彙整資訊、撰寫長報告；以 RL on research-task verifiers 訓練。
- **GPT-5 / 5.2 / 5.5** (2025 Aug–Dec)：「Steerable, collaborative」agentic mode；SWE-bench Verified 74.9%、Aider Polyglot 88%；GPT-5.5 在 agentic coding bench 達 82.7%。

- **Open problem**：UC Berkeley CRDI 在 2026 Apr 公開能自動 reward-hack 全部八個主流 agent benchmark 的攻擊 agent — 整個評估體系正在被質疑。

3.1.2 Anthropic Agent Stack

- **Claude Computer Use**（2024 Oct）：以 screenshot 為輸入、tool API 為輸出的 GUI 控制。
- **Model Context Protocol (MCP)**（2024 Nov）：開放的 JSON-RPC 2.0 標準（設計受 LSP 啟發），讓 LLM 以統一 schema 連結工具與資料源。到 **2025 Dec** 月下載量達 **9700 萬**；Anthropic 已將其捐贈給新成立的 Linux Foundation Agentic AI Foundation（與 Block、OpenAI 共同創立）¹⁹。
- **Claude Code**（2024–2026）：terminal-resident 的 agentic coding 工具，採 single-threaded master loop + 細粒度 permission system + 工具層（Read / Glob / Grep / Edit / Bash）；新版引入 multi-agent 分解（planner / executor / debugger / validator）。
- **Claude Skills**（2025）：以「Skill 包」（檔案集合）為單位的能力註冊與調用。

3.1.3 Google Agent Stack

- **Project Astra**（2024 Dec → I/O 2025）：跨裝置（手機 + 原型眼鏡）的 multimodal 助理，session memory 約 10 分鐘。
- **Project Mariner**（2024 Dec → I/O 2025）：Gemini 2.5 Pro 驅動的 web agent，可同時跑 **10 個 parallel task**，整合進 AI Mode 做預訂、查詢。
- **Gemini 2.5 Pro / Deep Think**：擴充為「world model」做 planning 與 imagined-experience simulation。

3.1.4 Orchestration Frameworks

框架	模型	特色	適用
LangGraph v1.0（2025）	任意	Stateful graph + conditional edges；LangChain 預設 runtime	產業 audit 與穩定性
CrewAI	任意	Role-based crews；2025 取得 \$18M A 輪，宣稱 60% 財富 500 大使用	中型企業 workflow
Microsoft Agent Framework （2025 Oct 由 AutoGen 與 Semantic Kernel 合併，2026 Q1 GA）	任意	GroupChat conversational pattern	Azure 生態
AutoGPT / BabyAGI lineage	GPT-4 系列	概念先驅，已大致被 graph-based 框架取代	教學示範

3.2 RLHF / RLAIF — 對齊的演算法

3.2.1 直接偏好最佳化（DPO 系列）

- **DPO**（Rafailov et al., NeurIPS 2023 ²⁰）：用一個閉式映射把 reward 模型「吸收」進 policy，pairwise preference loss 可直接訓 policy — 不需要 RM、不需要 online sampling，匹配甚至超越 PPO-RLHF。

- **IPO** (Azar et al., AISTATS 2024)：用 bounded objective 取代 Bradley-Terry，修正 DPO 在 deterministic preference 上的 unbounded margin 發散，OOD 泛化更好。
- **KTO** (Ethayarajh et al., ICML 2024 spotlight²¹)：用 prospect theory 的 utility maximization，只需要「好 / 壞」二元標籤，標註成本遠低於 pairwise；在大模型上匹配 DPO。

3.2.2 Constitutional AI / RLAIIF

- **Constitutional AI** (Bai et al., Anthropic 2022²²)：兩階段 — (1) SL：以「constitution」為原則做 self-critique + revise；(2) RL：以 AI 生成的偏好標籤訓練偏好模型再做 RLHF。「**RLAIIF**」一詞由此而來。
- **RLAIIF vs RLHF** (Lee et al., Google, ICML 2024²³)：在 summarization、helpful + harmless dialogue 上 RLAIIF $\approx$ RLHF；AI labeler 大小不必超過 policy；**d-RLAIIF** (off-the-shelf LLM 直接給 reward，跳過 RM) 反而打敗 canonical RLAIIF。
- **Open problem**：相同家族 labeler 帶來的 sycophancy；對抗 prompt 下 constitutional principles 仍易崩 (「Constitution or Collapse?」arXiv:2504.04918)。

3.2.3 推理 RL — PRM / ORM / RLVR

- **Process Reward Models (PRM)** (Lightman et al., OpenAI ICLR 2024²⁴)：每一步給 reward 比只給最終結果 (ORM) 更有效；釋出 PRM800K (800k step-level 標籤)，在 MATH 子集達 78% 解題率。
- **DeepSeek-R1** (Guo et al., **Nature** 2025, 首篇 peer-reviewed LLM **Nature**²⁵)：純 RL with **verifiable rewards** — 數學以符號驗證、程式碼以單元測試。提出 **GRPO** (Group Relative Policy Optimization) 取消 critic、以 group 內 K 個樣本的 advantage 標準化取代之；以 ~147K H800 GPU-hr (比 o1 級便宜約 10 倍) 達到 o1 等級推理能力。
- **OpenAI o1 / o3** (2024 Sep / 2025)：RL 訓練 long-CoT；提出 **inference-time scaling law** — 表現同時隨 train-time RL 計算與 test-time thinking budget 增長。
- **RLVR 理論** (arXiv:2506.14245)：RLVR 不只是提高「能 sample 出正確 CoT 的機率」，而是擴展推理邊界；提出 CoT-Pass@K 指標。
- **Open problem**：reward over-optimization；narrow verifier；數學/程式碼 RL 難以 transfer 到開放性推理 (「Crossing the Reward Bridge」arXiv:2503.23829)。

3.3 Tool-using / Function-calling Agents

- **ReAct** (Yao et al., ICLR 2023²⁶)：把 reasoning 與 acting 交錯生成 (Thought → Action → Observation loop)，是後續所有 agent loop 的 baseline。
- **Toolformer** (Schick et al., NeurIPS 2023)：自監督地在 pretraining 中插入 API call token，學「何時該叫 tool」。
- **Gorilla** (Patil et al., NeurIPS 2024²⁷)：微調 LLaMA 在 1600+ API 中做 API 選擇；衍生 Berkeley Function Calling Leaderboard (BFCL v4，含 agentic / multi-hop / error recovery 多項)。
- **ToolACE** (ICLR 2025)：自動合成 tool-use 資料的 pipeline，BFCL SOTA。

3.3.1 Agent Benchmarks

Benchmark	任務數 / 範圍	當前 SOTA	特性
SWE-bench / Verified (ICLR 2024)	2,294 真實 GitHub issue	~75–82% (GPT-5/5.5)	Execution-based
GAIA (Meta, ICLR 2024)	多工具複合題	--	單一 ground truth
AgentBench (ICLR 2024)	8 環境 (OS/DB/KG/...)	--	廣 coverage
WebArena (ICLR 2024)	812 長時域 web task	14.4% → 61.7% (IBM CUGA, 2025)	真實 web 環境
OSWorld (NeurIPS 2024)	369 跨 app real-OS task	~60pt 人類落差	真實 OS
τ-bench (Sierra/Yao, 2024)	Reliability : pass ^k	--	揭露一致性問題

Open problem : UC Berkeley CRDI 2026 Apr 公布能自動破解全部八個主流 benchmark 的 reward-hacking agent — 整個評估典範面臨重建。

3.4 Multi-Agent Collaboration

- **Generative Agents** (Park et al., Stanford, UIST 2023 ²⁸) : 25 個 simulacra 在沙盒小鎮中以「observe + reflect + plan + 記憶流」互動，產生 emergent 社會行為（自發辦派對、傳八卦）。
- **CAMEL** (Li et al., NeurIPS 2023) : 「inception prompting」讓兩個 LLM 扮演 user / assistant 對話自我引導；生成 AI Society + Code datasets。
- **MetaGPT** (Hong et al., ICLR 2024 oral ²⁹) : 把軟體開發 SOP 指派給 agent role (PM / Architect / Engineer / QA)，釋出時程式碼 Pass@1 達 85.9 / 87.7。
- **ChatDev** (Qian et al., ACL 2024) : CEO / CTO / Programmer / Tester 的「waterfall 軟體公司」。
- **AutoGen** (Wu et al., ICLR 2024 workshop / COLM 2024) : 比 MetaGPT/ChatDev 更靈活的 GroupChat。

3.4.1 MARL (classical multi-agent RL)

- 2024 surveys (Sci. Direct S2949855424000042、arXiv:2412.21088) : 合作型 MARL 仍由 CTDE 家族 (QMIX / MAPPO) 主導；open problem 是 curse of dimensionality、non-stationarity、跨異質 reward/state/action 空間的 multi-task 規模化。
- **LLM agent + MARL** 的融合是個 **open frontier** — LLM agent 缺乏 MARL 的 credit assignment 形式化；MARL 缺乏 LLM agent 的泛化能力。

3.5 Foundation Agents / Self-improving Agents

- **SIMA** (DeepMind, 2024 Mar ³⁰) : 跨多款商業 3D 遊戲用 keyboard/mouse 通用介面操作的 generalist agent ; 以 language instruction 觸發 ; 首次達 31% 人類相對成功率。 **SIMA 2** (2025 Nov) : 用 Gemini 驅動, 成功率約 v1 的兩倍, 在訓練過環境近人類, 能 transfer 到未見過遊戲。
- **Magma** (Microsoft, CVPR 2025 ³¹) : foundation model 同時處理 digital UI 與 physical robot ; 提出 Set-of-Mark (SoM) 做 action grounding、Trace-of-Mark (ToM) 做 action planning, 在 UI navigation 與 robot manipulation 上 SOTA。
- **STaR** (Zelikman et al., NeurIPS 2022) : Bootstrap rationales — 生成 → 以正確答案濾 → fine-tune ; 所有後續 rationale RL 的祖先。
- **Self-Rewarding LMs** (Yuan et al., Meta/NYU, ICML 2024 ³²) : 模型同時當 generator 與 LLM-as-judge, 做迭代 DPO ; instruction following 與 reward giving 一起進步。
- **Meta-Rewarding LMs** (Wu et al., arXiv:2407.19594) : 再加一層 meta-judge 降低 judge bias。
- **Open problem** : 多輪 self-iteration 後的 reward model collapse ; 缺外部 verifier 時的 grounding。

3.6 Memory Systems

路線	代表	優勢	限制
Long context (1M+ tokens)	Gemini 1.5/2.5、Claude 200K–1M	簡單、不需新架構	推論貴 ; needle-in-haystack 深度效能下滑
RAG	Llama-Index、LangChain RAG	標準外部知識 baseline	multi-hop / temporal update 表現差
MemGPT / Letta (Packer et al., 2023 ³³)	OS-style 階層記憶 (core / recall / archival), LLM 用 page-in/out 函式	長文件 QA、多 session chat	需手刻 paging 邏輯
Generative-agent memory (Park 2023)	Memory stream + recency/importance/relevance retrieval + reflection	模擬 episodic memory	規模化困難

Open problem : 缺乏 episodic vs semantic 的標準 benchmark ; long-context scaling law vs 結構化檢索的 trade-off 未解 ; 遺忘 / consolidation 政策無共識。

3.7 DRL 與 Agentic AI 的整合

3.7.1 DRL 在 LLM agent 中的角色

1. 對齊 (**alignment**) : PPO/GRPO + RLHF/RLAIF/RLVR 是把 base model 變成 helpful + harmless + honest 助理的核心。
2. 推理 (**reasoning**) : o1 / DeepSeek-R1 證明 RL 能放大 LLM 內隱的推理路徑分布, **inference-time compute** 與 **train-time RL** 是新 **scaling axis**。

3. 規劃 (**planning**) : MuZero-style 的 model-based planning 開始與 LLM 結合 (如 Tree-of-Thoughts、AlphaProof)。
4. 工具使用 (**tool use**) : 偏好 / 結果 reward 訓 agent 選對工具 (Gorilla, ToolACE)。
5. 多 agent 競合 : marketplace / debate / negotiation 用 self-play 風格 RL 訓練。

3.7.2 規劃與決策機制

- 靜態 **chain-of-thought** : 最簡單, 但無法回頭。
- **Tree-of-Thoughts / Graph-of-Thoughts** : 明確 search, 可重訪 node。
- **ReAct loop** : 感知 + 行動交錯, 每一步看到 env feedback。
- **MCTS-style with learned value** (如 AlphaProof) : LLM 提案、value model 評分、search expansion。
- **Plan-then-execute** : 先全局規劃再分配給 sub-agent (CrewAI、MetaGPT)。

3.7.3 人類回饋與對齊

從 InstructGPT 開始, **RLHF** → **DPO** → **KTO** → **RLAIF** → **RLVR** 這條演化路徑反映兩個趨勢 : (a) 降低人類標註成本 (pairwise → binary → AI labels → verifier) ; (b) 轉向可驗證信號 (從主觀偏好到客觀 ground truth)。下一步是 **process-aware reward** 與 **verifier** 之外的泛化 (如何把數學 RL 學到的推理 transfer 到開放性問題)。

3.7.4 通往 AGI 的方向

2025–2026 各家路線收斂在三條軸線上 : - 通用性 (**generality**) : foundation agent 跨任務、跨 embodiment、跨 modality (GR00T、Magma、SIMA 2、Gemini Robotics)。- 自主性 (**autonomy**) : long-horizon agent 能維持多小時/多天目標, 不需要人類每步監督 (Deep Research、Project Mariner)。- 可驗證學習 (**verifiable learning**) : 以 RL on verifiable rewards 持續 self-improve (DeepSeek-R1、Self-Rewarding LMs)。

Critical view. 雖然進展驚人, 但 (1) **reliability gap** 仍大 (τ -bench 顯示重複跑同一任務的一致性低)、(2) **benchmark gaming** 威脅整個 evaluation 典範、(3) memory 沒有共識解、(4) 真正的 long-term autonomous self-improvement 仍未被 demo。AGI 不是「規模再大一點」就會到, 當前 agentic AI 距離真正的 autonomous artificial general intelligence 仍有顯著 gap。

Part 4 — DRL Applications in Different Research Domains

本章選擇三個 2024–2026 進展最快、最能反映 DRL 與 foundation models 整合的應用領域, 逐一分析 : (a) 問題定義、(b) 為什麼 **DRL** 有用、(c) **SOTA** 系統、(d) 代表論文與分析、(e) 資料集 / **simulator**、(f) 未來方向。

4.1 Robotics 與 Embodied AI

4.1.1 問題定義

Robotics 涵蓋 humanoid locomotion、bimanual manipulation、mobile manipulation、HRI 等子任務。Embodied AI 進一步要求 agent 在具身（embodied）的感知 → 推理 → 行動迴圈中與環境互動，常見任務是 navigation、object rearrangement、language-instructed manipulation。這是一個 high-dim continuous control + partial observability + long horizon + sparse reward 同時存在的任務家族。

4.1.2 為什麼 DRL 有用

- 高維連續控制：對 humanoid 而言一次決策要輸出 30–60 維 torque/joint position，超出傳統 MPC 即時可解的範圍；off-policy DRL（SAC、TD3）與 on-policy（PPO + 大量並行 envs）能直接 end-to-end 學。
- 泛化性：用 randomization 訓出的 policy 可以橫跨數千個變體；imitation learning 雖然準，但對新場景的 fallback 不如 RL。
- 與 foundation model 互補：VLM 提供高層 semantic（這是什麼物件、目標在哪），RL/IL 提供低層 reactive control（如何接觸、力道多大）。

4.1.3 SOTA 系統（2024–2026）

類別	代表系統	年份	機構	一句話
VLA Foundation Model	OpenVLA ¹⁸	2024	Stanford+Berkeley+TRI+DeepMind+PI	第一個 open-weights VLA；7B params；+20.4% over Diffusion Policy
	$\pi 0$ / $\pi 0$ -FAST / $\pi 0.5$ ³⁴	2024–25	Physical Intelligence	flow-matching 50 Hz； $\pi 0.5$ 能清掃「完全沒見過的廚房」
	Helix ³⁵	2025	Figure AI	dual-process S2 (7-9 Hz VLM) + S1 (200 Hz visuomotor)
	GR00T N1 ³⁶	2025	NVIDIA	首個 open humanoid FM；63.9 ms latency on L40
	Gemini Robotics / 1.5 ³⁷	2025	Google DeepMind	驅動 Apptроник Apollo；加入 agentic tool use
	Magma ³¹	2025	Microsoft	同時做 digital UI + physical manipulation
Imitation / Diffusion Policy	Diffusion Policy ³⁸	2023	Columbia	conditional denoising 處理 multimodal action；+46.9% over prior SOTA
	DP3 (3D) ³⁹	2024	Stanford	point cloud + diffusion；40 demo 達 85% real success
	Mobile ALOHA	2024	Stanford	\$32k 雙臂全身 teleop；做飯、清潔
	Octo / CrossFormer ⁴⁰	2024	Berkeley	開源 800k OXE transformer；單一 model 多 embodiment
RL on Real Robot	Eureka / DrEureka ⁴¹	2024	NVIDIA+UPenn	LLM 寫 reward function；80%+ 任務超越人工

類別	代表系統	年份	機構	一句話
	SERL ⁴²	2024	Berkeley	real-world RL 25–50 分鐘訓出 PCB / 線材組裝
Humanoid WBC	HumanPlus	2024	Stanford	RGB 影片 shadowing
	OmniH2O ⁴³	2024	CMU LeCAR	VR + voice + RGB ; GPT-4 autonomy
	ASAP ⁴⁴	2025	CMU+NVIDIA	sim-pretrain + delta-action residual 補 sim2real
快速 RL on G1	arXiv:2512.01996	2025	UC Berkeley	RTX 4090 上 15 min 訓出 Unitree G1 locomotion

4.1.4 代表性論文分析

OpenVLA (2024) 重要性在於打破 **RT-2** 的閉源壟斷。Stanford 用 Llama 2 + SigLIP/DINOv2 在 970k Open X-Embodiment trajectory 上訓出 7B model，並把 action tokenize 成 7-DoF 離散 token。LoRA fine-tune 即可在新 embodiment 上適配，4-bit quantization 仍保留性能。**Critical view**：5 Hz inference latency 對高頻 dexterous control 仍是瓶頸（這也是後續 dual-system 出現的原因），且離散 action token 對精細力控限制大。

$\pi 0.5$ (Apr 2025) 是首個能在「完全未見過的真實住家」做開放性任務（如清潔廚房 / 整理臥室）的 VLA — 不是 demo 而是反覆 trial 的成功率。技術上靠 hierarchical reasoning（用 VLM 規劃 sub-goal）+ heterogeneous co-training（web data + verbal instructions + 多機器人資料）。**Critical view**：成功率仍非可商業部署等級（特定任務 60–80%）；長時域失敗仍主流。

Helix (Feb 2025) 的工程意義很大：在 embedded GPU 上跑完整 dual-process 架構，**S2 跑 7–9 Hz 做語意 / 規劃**、**S1 跑 200 Hz 做 visuomotor**，控制 Figure 02 的 35 DoF — 證明 dual-system 不是學術點子，能放到出貨機上。

4.1.5 資料集與 Simulator

- **資料**：Open X-Embodiment（1M+ traj, 22 embodiments, 2024 Google + 21 機構聯盟）、DROID（76k demos, 564 scenes, RSS 2024）、Mobile ALOHA dataset、BridgeData-V2、RoboCasa 100 廚房任務。
- **Simulator**：Isaac Lab（GPU end-to-end，2025 推 4.x）、Isaac Sim、MuJoCo MJX（JAX-native, differentiable, 2025）、ManiSkill3（SAPIEN backend, 2024）、Habitat 3.0（室內 navigation）、RLBench、CALVIN、LIBERO、Newton（2025 NVIDIA + Google + Disney）。
- **數字**：Isaac Lab 在 8× RTX Pro 6000 跑 humanoid RL 達 **2M env step / sec**，manipulation 達 900k FPS。

4.1.6 未來方向

1. **Dual-system architecture** 標準化：S2 (VLM, 5–10 Hz) + S1 (visuomotor, 100–200 Hz) 已成 Helix / GR00T N1 / Gemini Robotics 1.5 共識。
2. **World-model + VLA 整合**：V-JEPA 2、1XWM 嘗試把 world model 變成 robot 的 imagination engine。
3. **Sim2Real with differentiable physics**：Newton + MJX 推動「梯度可穿過 simulator」。
4. **Verifier-driven robot RL**：類似 RLVR 在 LLM 推理上的成功，下一步是怎麼為 manipulation 設計 reliable verifier。
5. 資料 **scaling**：1M trajectory 仍只是 LLM 訓練資料的 < 1%，需要更便宜的 teleop / 影片監督路線 (HumanPlus、ASAP)。

4.2 Game AI

4.2.1 問題定義

Game AI 涵蓋 perfect-information board games (圍棋、西洋棋)、imperfect-information games (撲克、Diplomacy)、realtime competitive video games (StarCraft、Dota、GT 賽車)、open-world creative games (Minecraft)。共同挑戰：長時域 **credit assignment**、多 **agent** 競合、龐大狀態空間、稀疏 **reward**。

4.2.2 為什麼 DRL 有用

- Self-play 提供 **automatic curriculum** — 對手隨自己變強，永遠在學區邊緣。
- 棋類遊戲的 **可驗證 reward** (勝負) 讓 RL 可以遠超 supervised learning (後者受限於人類 demonstration 的上限)。
- 多 agent setting 中 **Nash equilibrium** / 子博弈完美 **equilibrium** 提供漂亮的理論 grounding。

4.2.3 SOTA 系統 (Lineage Map)

2016 AlphaGo (Nature) – human-data + RL self-play + MCTS
2017 AlphaGo Zero (Nature) – tabula rasa
2018 AlphaZero (Science) – generalized to chess/shogi
2019 AlphaStar (Nature) – StarCraft II Grandmaster, League training
2019 OpenAI Five (Dota 2) – PPO + 128k CPU
2020 MuZero (Nature) – learned latent dynamics
2022 Cicero (Science) – Diplomacy + LLM dialogue
2022 DeepNash (Science) – Stratego, model-free Nash (R-NaD)
2022 Gran Turismo Sophy (Nature) – beat human champions, QR-SAC
2023 Voyager (NeurIPS) – Minecraft LLM agent, no gradient RL
2024 AlphaGeometry (Nature) – IMO geometry
2024 SIMA (DeepMind) – 9 commercial 3D games, generalist
2024 Tearing-instability avoidance (Nature) – DIII-D plasma control
2025 DreamerV3 (Nature) – first to mine diamonds **from scratch**
2025 SIMA 2 (Dec 2025) – Gemini-powered, self-improving in 3D
2025 AlphaProof + AlphaGeometry 2 (Nature, Nov) – IMO 2024 silver, 28/42

4.2.4 代表性論文分析

AlphaStar (Nature 2019) ⁴⁵. 第一次在 realtime imperfect-information game 中達 Grandmaster。架構結合 (a) 971k 人類 replay 的 supervised imitation、(b) transformer + pointer network、(c) **League training** — 主 agent 與 main-exploiter / league-exploiter 共同演化以避免 strategy cycling。 **Critical view**：APM 與 camera 限制曾受爭議；對 OOD 的人類策略仍脆弱；訓練成本約 \$10M。 啟示：純 RL on Pong 行得通，但要打複雜遊戲，supervised init + population-based 是必要的。

MuZero (Nature 2020) ⁸. 真正把 model-based RL 帶到實用層級。核心是 learning a **value-equivalent** latent dynamics model — 不必預測完整 next-state pixels，只預測「對 policy / value 有用」的潛在表徵。同一套架構打敗 AlphaZero（棋類）+ R2D2（Atari）。 **Critical view**：訓練成本高；對 stochastic env（如撲克）不擅長；模型本身非 interpretable。

DreamerV3 (Nature April 2025) ⁴⁶. 第一個用單一 **hyperparameter config** 解 150+ 任務的演算法，且首次「從零開始」（無 human data）在 Minecraft 挖到鑽石（~30M env steps）。技巧：Recurrent State-Space Model + categorical latents + symlog squashing — 處理不同 reward / observation scale 的關鍵。 **Critical view**：訓練 wall-clock 仍以天計；無法像 VPT 那樣利用 YouTube 影片。

AlphaProof + AlphaGeometry 2 (Nature Nov 2025) ⁴⁷. IMO 2024 拿銀牌（28/42 分），解了 6 題中的 3 題，包括 Q6（人類只有 5 位解出）。 **AlphaProof**：用 Lean formalize，AlphaZero-style RL 在百萬 auto-formalized 題目上訓練。 **AlphaGeometry 2**：神經符號 hybrid — LLM 提案輔助線，symbolic engine 驗證，比 v1 快 100×（IMO 2024 Q4 在 19 秒內解出）。 **Critical view**：combinatorics 仍未解；formalization 仍是瓶頸；難 transfer 到研究級數學。

SIMA 2 (Dec 2025) ⁴⁸. 用 Gemini 驅動的 self-improving 3D 遊戲 generalist。最關鍵的創新是 **self-play loop** 使用 **Gemini-generated reward** — 在新 env 不靠人類設計 reward，靠 LLM judge。約是 v1 兩倍成功率，trained env 接近人類。 **Critical view**：長時域規劃仍弱、精細運動仍弱、evaluation 限於 DeepMind 自選 game。

4.2.5 資料集 / Simulator

- **棋類**：AlphaZero 用自己的 self-play；無外部 dataset。
- **StarCraft / Dota**：人類 replay 資料集 + 自家 simulator。
- **Minecraft**：MineDojo（NeurIPS 2022）— simulator + 730k YouTube 影片 + 7k wiki page + 340k Reddit；VPT 用 contractor 標註 → IDM → 70k 小時 unlabeled video。
- **GT 賽車**：Sony PlayDriver simulator（內部）。
- **數學**：Lean 4 mathlib（百萬 formalized 定理）。

4.2.6 未來方向

1. **Foundation agent for games**：SIMA 2 的 self-improving loop 是雛形；目標是跨遊戲、跨遊戲類型的單一 model。
2. **Verifier-driven scientific game**：把 IMO 推理 transfer 到實際定理（millennium prize 級別）。
3. **Open-ended creative game AI**：Voyager 開頭、SIMA 2 接力，但「無限發明新任務」仍未真正解決。

4. 多模態 **game agents**：把語音 / 視訊 / 鍵盤 / mouse 視為通用 action space，向 SIMA / Helix-style 收斂。
-

4.3 AI for Science

4.3.1 問題定義

科學發現的核心活動：(1) 大空間搜尋（化合物、材料、theorem、algorithm），(2) 多 step decision under uncertainty（實驗該做哪個 / 該量哪個量），(3) 高成本驗證（合成、實驗、formal proof）。這恰好對應 RL 的核心結構：long-horizon decision、稀疏但可驗證的 reward。

4.3.2 為什麼 DRL 有用

- **Verifiable reward**：許多科學問題有可驗證的成功訊號（化合物的 binding affinity、formal proof 的 Lean check、algorithm 的執行時間），這正是 RLVR 的理想場景。
- 跟人類昂貴的試錯比優勢：模擬器內 trial 便宜（毫秒），把人類數年研究壓縮到數天。
- **Foundation model + RL outer loop** 已成主流範式：LLM 提案、verifier scoring、RL 更新 — 與 AlphaProof / FunSearch / AI-Scientist 一脈相承。

4.3.3 SOTA 系統（不限於純 DRL，但 RL 是核心驅動）

子領域	代表系統	期刊	年份	RL 角色
蛋白質結構	AlphaFold 2/3	Nature 2021/2024	2021/24	非 RL（diffusion + attention），但下游設計用 RL
De novo 蛋白質設計	RFDiffusion ⁴⁹	Nature 2023	2023	生成 diffusion + 下游 RL fine-tune ProteinMPNN
化合物設計	REINVENT 4 ⁵⁰	J. Cheminf. 2024	2024	PPO 直接 optimize multi-property reward （QED, SAS, docking）
晶體 / 材料	GNoME ⁵¹	Nature 2023	2023	Active learning（RL-adjacent）
自主合成	A-Lab ⁵²	Nature 2023	2023	LLM 抽 recipe + 熱力學 active learning
代數演算法	AlphaTensor ⁵³	Nature 2022	2022	純 AlphaZero MCTS 找 tensor decomposition
數學發現	FunSearch ⁵⁴	Nature 2023	2023	LLM 提 mutation + score-based selection
	AlphaProof	Nature 2025	2025	AlphaZero-style RL on Lean proof tree
	AlphaGeometry 2	Nature 2025	2025	LLM 提 aux 線 + symbolic verifier
演算法	AlphaDev ⁵⁵	Nature 2023	2023	AlphaZero 在 assembly instruction space； merged into LLVM libc++
電漿控制	DIH-D Tearing avoidance ⁵⁶	Nature 2024	2024	RL 即時調磁場避開 tearing instability
晶片設計	AlphaChip ⁵⁷	Nature 2021 + addendum 2024	2021/24	PPO + GNN 做 floorplan；部署在 TPU
自主科學家	Coscientist ⁵⁸	Nature 2023	2023	GPT-4 agent + 真實實驗室；做 Pd-catalyzed coupling
	AI Scientist v2 ⁵⁹	arXiv 2025	2025	首次 AI-authored paper 通過 ICLR 2025 workshop peer review

4.3.4 代表性論文分析

AlphaDev (Nature 2023) ⁵⁵. 真正影響日常 computing — DeepMind 把 AlphaZero 套用到 assembly instruction game，找到 short-sequence sort 比現有 libc++ 快 **70%** 的演算法，**merged into LLVM libc++**，每天執行數兆次。Latency reward per-instruction，correctness 用 test-equivalence。**Critical view**：只在 short branchless kernels 有效；不能 generalize 到任意演算法。**意義**：是少數能說「我的 RL 改變了基礎建設」的案例。

DIII-D Tearing-instability avoidance (Nature Feb 2024) ⁵⁶. 在真實 tokamak 上即時跑 RL 控制磁場，避免電漿 disruption（這是 fusion 商用的最大障礙之一）。能 compete 古典 PID/MPC 是因為 plasma dynamics 太高維。**Critical view**：sim2real 仍需 per-tokamak retraining；ITER 級別未證；reward 含 engineered terms 不確定通用性。

AI Scientist v2 (Apr 2025) ⁵⁹. Sakana AI 的 v2 用 agentic tree search + experiment manager。首篇 AI 全自動撰寫的 paper 通過 **ICLR 2025 workshop peer review**。雖然有 arXiv 2502.14297 對 v1 的尖銳批評（弱 literature synthesis、表面 novelty、shallow ablation），這仍代表一個門檻被跨過。**意義**：autonomous research agent 從概念變成可量化驗證的階段。

GNoME + A-Lab (Nature 2023, 連續發表) ^{51 52}. GNoME 用 GNN + active learning 發現 2.2M 晶體 candidate，A-Lab 用自主機器人 17 天合成 41/58 個 GNoME 推薦。**Critical view**：2024 年 Cheetham & Seshadri 批評許多 GNoME "新材料" 其實是 rediscovery 或 composition duplicate；A-Lab 的 phase ID 也被質疑。但這仍是 **RL + 自主實驗室「閉環」** 的最完整 demo。

4.3.5 資料集 / Simulator

- 蛋白質：PDB（19 萬+ 結構）、UniProt 序列、ESM-3 訓練用 2.78B proteins。
- 化合物：ZINC, ChEMBL, GuacaMol, MOSES benchmark。
- 材料：Materials Project, ICSD, OQMD；GNoME 的 release。
- 數學：Lean 4 mathlib, IMO past papers, MathPile, ProofNet。
- 演算法：CodeContests（AlphaCode），LLVM IR, GEMM benchmark。
- 科學文獻：S2ORC, OpenReview replicas。

4.3.6 未來方向

1. **Closed-loop autonomous lab**：A-Lab 是雛形，下一步是「LLM 提假設 → RL 規劃實驗 → robot 執行 → 解讀資料 → 更新假設」全自動。
2. **RL on formal verifier**：AlphaProof 證明這條路可行，把它擴到生物學（如 PathML proof）、物理（symbolic regression）。
3. 避免 **reproducibility crisis**：AlphaChip / GNoME 的爭議警示，AI for Science 需要 open benchmark 與 independent replication，不只是 Nature 封面。
4. 多 **modal scientific agent**：未來 agent 同時看 paper PDF、跑 simulation、操作真實 instrument，朝 Coscientist + AI Scientist v2 整合。

Part 5 — SOTA Research Trends (2025–2026)

把 Part 1–4 收斂起來看，可以找出 十條 2025–2026 真正在推進 DRL 與 agentic AI 邊界的研究 trend。本章對每條 trend 解釋 為什麼重要 / 當前限制 / 未來方向。

5.1 Embodied AI

為什麼重要。從 ChatGPT 之後，業界普遍相信 AI 的下一個 frontier 在 **physical world**——能拿東西、能在你廚房做菜、能去 Mercedes 工廠搬箱子。2025 年的代表事件：- Figure 03 (Oct 2025) 以 12k units/年產能設計；- Apptronik Apollo 拿 \$520M, \$5B 估值；- 1X NEO 開放消費市場（\$499/月訂閱）；- Tesla Optimus Gen 3 demo kung-fu / 烹飪。

當前限制。(1) 資料稀缺：Open X-Embodiment 1M+ trajectory 在 LLM 資料尺度上只是「打底」；(2) 長時域 **reliability**：示範影片漂亮，反覆 trial 的 pass^k 仍低；(3) **安全認證**：沒有 humanoid 取得 unstructured home deployment 的 formal safety cert。

未來方向。大規模 teleop fleet（特斯拉、Figure）+ 人類影片監督（HumanPlus、V-JEPA 2）+ differentiable simulator + dual-system architecture，是接下來 3–5 年的工程主軸。

5.2 World Models

為什麼重要。World model 是讓 agent 「在腦中想像 → 規劃 → 行動」的核心。2024–2025 三個並行突破：- **DreamerV3** (Nature April 2025)：第一個用單一 hyperparameter 跨 150+ 任務、且從零挖到 Minecraft diamond 的演算法。- **Genie 2 → Genie 3** (2024 Dec / 2025 Aug)：DeepMind 從 2D playable 推到 720p 24 fps 多分鐘 consistent 的 interactive world，已釋出 "Project Genie" (2026 Jan)。- **V-JEPA 2** (Meta Jun 2025)：1.2B params 在 1M 小時影片 + 62 小時 robot data 上訓練，zero-shot pick-and-place 65–80%；比 NVIDIA Cosmos 快約 30×。

當前限制。Sora-style video model 的 cause-and-effect 仍弱（會 hallucinate 不合物理）；Genie 3 有 10 秒級 short-term memory；V-JEPA 走 embedding-space prediction，對 fine control 不夠直接。

未來方向。World model + RL outer loop（用 world model 當 imaginary env，把 RL data efficiency 推上幾個量級）是 2026–2027 預期突破點；differentiable physics（Newton、MJX）會強化這條路。

5.3 Multi-agent RL（MARL 與 LLM agent 的融合）

為什麼重要。真實場景幾乎沒有 single-agent — 自駕、生產線、市場、社交都是 multi-agent。經典 MARL（QMIX、MAPPO、CTDE 家族）解 cooperative 與 zero-sum 已有成果；但 **LLM 多 agent**（MetaGPT、CrewAI、AutoGen）給了新的可能性：以自然語言為通訊媒介、以 role 為分工。

當前限制。(1) MARL 對於異質 reward / state / action space 的 multi-task 擴展仍開放；(2) LLM agent 群組缺少 MARL 嚴格的 credit assignment；(3) 兩條路線還沒有像 **LLM + RL** 那樣的成功融合典範。

未來方向。Population-based training（AlphaStar 風格）套到 LLM agent；RLAIF 形式的 group reward；MARL 中嵌入 LLM 提案層。SIMA 2 的 group self-play 是早期 prototype。

5.4 Offline RL

為什麼重要。機器人、醫療、金融、自駕的真實場景不能 **online try-and-error**，只能從歷史 dataset 學。Offline RL（BCQ / CQL / IQL / Decision Transformer）讓「從 log 學 policy」變成可行。

當前限制。(1) Distribution shift 仍是核心難題；(2) 上限受限於 behavior policy 覆蓋；(3) 缺少 **well-known benchmark** 之外的真實成功案例——D4RL/Atari/RoboMimic 之外，產業界 deployment 仍少。

未來方向。Offline-to-online fine-tuning（先 offline、再小量 online 修正）、與 VLA fine-tuning 結合（用 RL 修 imitation policy 的 reliability tail）。

5.5 Diffusion Policy

為什麼重要。機器人示範資料天生 multimodal（同個 sub-goal 可有多種完成方式），Gaussian policy 容易塌縮。Chi et al. (RSS 2023, IJRR 2025) 的 **Diffusion Policy** 用 conditional denoising 自然表達多峰，+46.9% over 先前 SOTA across 12 任務。

當前限制。(1) 推論 latency 高（多步 denoising）；(2) cross-embodiment transfer 不佳；(3) 對 reflective / transparent objects（DP3 依賴 depth）效果差。

未來方向。Distillation 把多步 diffusion 壓成 single-step（consistency models）；flow matching 路線（ π_0 ）；與 VLA 整合（OpenVLA 仍是 discrete token，DP-style continuous output 是改進空間）。

5.6 Foundation Agents

為什麼重要。從 specialist policy 轉向「一個 **model** 跨多任務、多 **embodiment**、多 **modality**」。代表：- **Magma** (CVPR 2025)：同時做 UI 與 robot manipulation；- **GR00T N1**：首個 open humanoid foundation model；- **SIMA 2**：跨多款 3D game 的 generalist；- **Voyager**：用 LLM 當 in-context RL 的 outer loop；- **Octo / CrossFormer**：多 embodiment robot policy。

當前限制。(1) Per-task specialist 仍打贏 foundation agent（trade-off 仍在）；(2) 缺乏 well-defined evaluation 跨領域；(3) 訓練成本急速上升。

未來方向。統一的 action space tokenization（OpenVLA 風格的 discrete token、或 π_0 的 flow continuous）；more compute → emergence；foundation agent + 在 deployment 端做 RL personalization。

5.7 Robotics Foundation Models

為什麼重要。與 5.6 重疊但聚焦 robotics — 2024–2025 是 robotics 自己的「BERT moment」：- **OpenVLA** (2024)：第一個 open VLA。- **π_0 / $\pi_0.5$** (2024–25)：高頻 continuous control 突破。- **Helix** (Feb 2025)：dual-system 上 embedded GPU。- **GR00T N1** (Mar 2025)：NVIDIA open humanoid FM。- **Gemini Robotics 1.5** (Sep 2025)：加入 agentic tool use；部署 Apollo。

當前限制。(1) Latency vs capability tradeoff（VLA 越大越聰明，越大延遲越高）；(2) Data scarcity；(3) Per-embodiment generalization 仍弱；(4) LIBERO-PRO 揭示 benchmark memorization 問題。

未來方向。Dual-system 標準化、heterogeneous co-training（web + verbal + 多 robot）、與 world model 整合。

5.8 Sim2Real Transfer

為什麼重要。真實機器人訓練昂貴、危險、慢；模擬器訓便宜、安全、快百萬倍。**Sim2Real gap** 是這條 pipeline 的決勝點。

2025 重要事件：- **Newton physics engine** (NVIDIA + Google + Disney, GTC 2025)：open-source、GPU-accelerated、differentiable，**humanoid 70×** 加速、**in-hand manipulation 100×**。- **MuJoCo MJX + Playground** (Feb 2025)：JAX-native，sim2real 到 6 個 robot 平台只花 < 8 週。- **DrEureka** (RSS 2024)：LLM 同時自動生成 reward 與 domain randomization 參數。- **ASAP** (2025)：sim-pretrain + delta-action residual model 補 sim2real gap，可達 agile parkour。

當前限制。Contact-rich / deformable / fluid 仍是 simulator 的短板；vision sim2real 仍需 photorealistic rendering（如 Cosmos、Omniverse）。

未來方向。Differentiable physics 直接讓 gradient 穿過 simulator；real-data residual learning（ASAP）；foundation video generator + RL（V-JEPA 2、Sora-style）。

5.9 Autonomous Scientific AI

為什麼重要。AI 不只是工具，而是科學家本身——這是 AGI 最具體的試金石。代表：- **Coscientist** (Nature 2023)：GPT-4 agent + 真實實驗室，做 Pd-catalyzed coupling。- **A-Lab** (Nature 2023)：自主合成 GNoME 提案的 41 個材料 / 17 天。- **AlphaProof + AlphaGeometry 2** (Nature 2025)：IMO 2024 銀牌（28/42）。- **AI Scientist v2** (Sakana, Apr 2025)：首篇 **AI-authored paper** 通過 **ICLR 2025 workshop peer review**。

當前限制。(1) Literature synthesis 仍弱；(2) Novelty 多為 surface-level；(3) Reproducibility crisis（GNoME / AlphaChip 都受 2024 critique）；(4) 缺少 cross-discipline 通用 verifier。

未來方向。封閉迴圈 hypothesis → experiment → analysis → next hypothesis 全自動；以 RL 在 verified scientific reward 上 self-improve；多 modality scientific agent（看 PDF + 跑 sim + 操 instrument）。

5.10 RL + Transformers

為什麼重要。兩條融合線：

A. Transformer 當 policy 架構（policy as sequence model）- **Decision Transformer / Trajectory Transformer**（NeurIPS 2021）- **Gato**（DeepMind 2022）：一個 transformer 玩 Atari + 寫字 + 操作 Sawyer 機械手 + 對話 - **Octo / OpenVLA / $\pi 0$ / GR00T**：robotics 領域全部 transformer-based

B. RL 訓練 Transformer policy（用 RL 做後訓練）- RLHF / RLAIIF / DPO / KTO 訓 LLM - RLVR（DeepSeek-R1 / o1）訓 reasoning model - GRPO（DeepSeek 提）取消 critic 降低訓練成本

當前限制。(1) Transformer 的計算成本對 high-frequency control 是負擔；(2) RL post-training 對 base model 的依賴強（pre-training 沒覆蓋的能力，RL 很難「無中生有」）；(3) 缺少 standardized RL+Transformer benchmark。

未來方向。Mamba/SSM 等線性序列 model + RL；長時域 token-by-token planning；test-time RL（在 inference 時繼續學）。

5.11 五條趨勢的交叉骨架

以上十條 trend 互相關聯，可以收斂成 五條主軸：

1. Foundation Models 與 RL 的融合

- LLM/VLM 提供 prior, RL 提供 alignment + reasoning

- 代表：RLHF, RLAIF, DPO, RLVR, GRPO; SIMA 2, AI Scientist v2

2. Embodied generalist agent

- Single model 跨 task / embodiment / modality

- 代表：OpenVLA, $\pi 0.5$, GR00T N1, Magma, Gemini Robotics 1.5

3. Sim2Real + Differentiable Physics

- GPU-accelerated photorealistic sim, 梯度可穿過 simulator

- 代表：Newton, MJX, Isaac Lab 4.x, DrEureka

4. World Model + Imagined Planning

- 在 agent 腦中模擬世界做 planning

- 代表：DreamerV3 (Nature 2025), Genie 3, V-JEPA 2

5. Verifier-Driven RL

- Formal proof / code execution / sim 作為可驗證 reward

- 代表：DeepSeek-R1, AlphaProof, AlphaGeometry 2, AlphaDev

這五條主軸構成 2025–2026 DRL 與 agentic AI 的「全景」，下一階段的突破很可能來自它們的交叉點（如 world-model RL on humanoid、verifier-driven scientific agent）。

Part 6 — Comparative Analysis

下面以五個代表性演算法做橫向比較。除了題目要求的 strength / weakness / sample efficiency / real-world usage 四維，再額外加入 **policy type**、**on/off policy**、**planning vs model-free**、典型 **wall-clock** 與是否仍是 **SOTA baseline** 等資訊以增加 analytical depth。

6.1 五法概覽表

$p\{2.4\text{cm}\}p\{3.0\text{cm}\}p\{3.0\text{cm}\}p\{1.7\text{cm}\}p\{3.0\text{cm}\}$

演算法	**類型**	**Strength**	**Weakness**	**Sample Eff.**	**Real-world Usage**
DQN	Value-based / off-policy / discrete	Atari benchmark 元祖、簡單、replay 與 target net 變所有 DQN 系列基石	$\max$ 造成 overestimation；連續動作不可用；訓練不穩	中（off-policy、replay）	推薦系統、廣告 bidding、教學示例；產業已大量被 Rainbow / PPO 取代
PPO	Policy-based / on-policy	實作簡單、超參魯棒、離散 / 連續通用； **LLM RLHF 事實標準**	on-policy 因此 sample-inefficient；需大量並行 envs；對 advantage normalization 等實作細節敏感	低	RLHF (GPT/Claude/Gemini)、Dota 2/OpenAI Five、自駕、Isaac Lab humanoid RL
SAC	Actor-critic / off-policy / continuous	連續控制 SOTA baseline；entropy 鼓勵探索；temperature 可自動調；穩	連續動作專屬；需要 twin critic ensemble；實作較 PPO 複雜	高	機械手、四足 / 人形 locomotion (Berkeley G1 15 min)、UAV、自駕；SERL 在真實機器上 25-50 分訓出 PCB 組裝
MuZero	Model-based / planning + RL	同一套架構解 board games + Atari；不需已知環境模型；極高 sample efficiency	工程複雜（MCTS + reps learning）；對 stochastic env 較弱；訓練成本高	高（learned model）	DeepMind 內部（video compression, search）、學術 model-based RL 標竿；產業少見
Decision Transformer	Sequence model（無 RL update）	把 offline RL 變為純 supervised；天然支援 multi-task return-conditioning；Transformer scaling laws	Stitching 差於 DP-based offline RL；對 OOD return target 敏感；需大量資料	中（offline，與資料量呈正比）	VLA 系列（Gato / Octo / OpenVLA / GR00T）的概念祖先；offline RL benchmarks

6.2 深度討論

6.2.1 為什麼 PPO 變成「萬用 baseline」？

PPO 並不是任一單一指標最好的演算法 — sample efficiency 不如 SAC、planning 不如 MuZero。但它在「實作 / 超參 / on-policy 規模化 / 離散+連續通用 / 對 reward scale 魯棒」五個維度同時及格。當你

的任務是 (a) sim 可以大量並行、(b) 需要 RL 對齊預訓 model（如 RLHF），PPO 幾乎是 default choice。它的最大競爭者是 **GRPO**（DeepSeek 提出，取消 critic 改用 group 標準化 advantage）— 在 LLM 推理 RL 上 GRPO 已逐漸取代 PPO。

6.2.2 為什麼 SAC 在連續控制這麼穩？

三個原因：(1) off-policy → replay → sample efficient；(2) twin Q + clipped target → 抑制 overestimation；(3) maximum entropy → 自然探索 + 對多模態最優解友善。SAC 在學術 benchmark 通常打贏 PPO，但 sim2real 部署時 PPO 仍常被選 — 因 PPO 在大規模 distributed 訓練更可預測、不易 critic 過擬合。

6.2.3 MuZero 為什麼沒在產業界遍地開花？

工程上 MuZero 很重 — 同時學三個 head（policy / value / reward）+ MCTS + representation。當 simulator 便宜（如 Atari、棋類），model-free 的 R2D2 / Rainbow / Muesli 已夠強；當需要 model-based 帶來的 sample efficiency，DreamerV3 反而更輕量、且在 Minecraft / DMControl 上 SOTA。**MuZero** 是漂亮的學術里程碑，但 **DreamerV3** 是更實用的工程繼承者。

6.2.4 Decision Transformer 開啟了什麼？

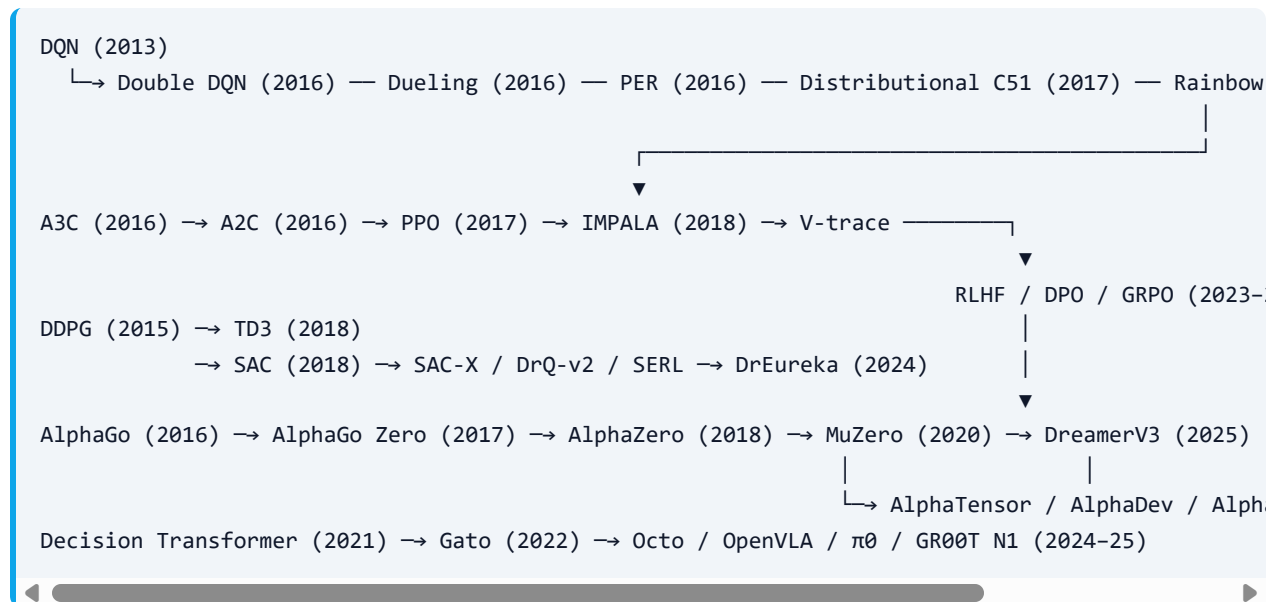
DT 證明「**offline RL** 可以是純 **supervised learning**」，把 RL 變成 sequence modelling，這個 framing 直接導致：- **Gato**（DeepMind 2022）：一個 transformer 跨 600+ 任務 - **Octo / OpenVLA / GR00T**：robotics VLA 全部 transformer-based、全部 IL-as-sequence-modeling

不過 DT 也有底層缺陷：缺乏 dynamic programming 的 **stitching** 能力（從 sub-optimal 段落拼最優），所以在純 offline benchmark 上常輸 CQL / IQL。實務上 DT-like 架構通常配 RL fine-tuning 補足。

6.2.5 哪種演算法選哪種任務？

任務情境	推薦演算法	理由
Atari / 離散決策	Rainbow DQN / Muesli	已是 model-free SOTA
連續控制（學術 benchmark）	SAC / TD3	sample efficient
大規模分散式（Dota/StarCraft）	PPO + League	簡單可預測
LLM RLHF / RLAIIF / RLVR	PPO / GRPO + DPO/KTO	事實標準
Board games / 棋類	AlphaZero / MuZero	self-play + MCTS
開放世界 (Minecraft / DM Control)	DreamerV3	單一 hyperparam 跨任務
Offline RL（醫療 / log）	IQL / CQL / DT	OOD-safe
Robot manipulation（demonstration 豐富）	Diffusion Policy / OpenVLA	multimodal continuous
Robot manipulation（real-world fast）	SERL (RLPD)	25–50 分鐘從零

6.2.6 趨勢圖



這張圖揭示 2025–2026 三條演化主線：(1) policy 從 model-free 走向 model-based / world-model 路線；(2) sequence modelling 已成為 robotics policy 的主流架構；(3) 所有 SOTA 演算法的「外圍」都被 LLM / foundation model 包覆。

Part 7 — GitHub / Open Source Ecosystem Survey

DRL 的迭代速度很大程度由開源 community 決定。本章梳理十個對 2024–2026 研究最有影響力的 GitHub 專案，每個都從 主要用途 / 支援演算法 / 優勢 / 弱點 / 研究 vs 產業適用 五個維度評估。

7.1 Stable-Baselines3 (SB3)

Repo. DLR-RM/stable-baselines3 — 12k+ stars。 **用途.** PyTorch 重寫的 OpenAI Baselines；學術界最被引用的 DRL library。 **演算法.** PPO, A2C, DQN, SAC, TD3, DDPG, HER。 **優勢.** 文件完整；**rl-baselines3-zoo** 提供 100+ 個 tuned hyperparameter；reproducibility 一流。 **弱點.** 不支援 distributed training；演算法保守，不追新 SOTA（無 DreamerV3 / MuZero）。 **適用.** 研究 **baseline**、教學、**prototyping** 是首選；產業大規模 deployment 換 RLlib。

7.2 Ray RLlib

Repo. ray-project/ray — 33k+ stars（整個 Ray ecosystem）。 **用途.** 建構在 Ray 上的 distributed RL framework；UC Berkeley RISELab → Anyscale 商業化。 **演算法.** 30+ algorithm（PPO, APPO, IMPALA, SAC, DQN, R2D2, DDPG, MARL 如 QMIX / MADDPG / PPO-Centralized 等）。 **優勢.** Scalability 一流，能從單機擴到上千 CPU/GPU；產業級維護；MARL 支援好。 **弱點.** 抽象多、API 學習曲線陡；2.x → 3.x 大改動讓社群一度抱怨穩定性；單機快速實驗反而不順手。 **適用.** 產業大規模 **DRL** 主流選擇；學術界與 SB3 互補。

7.3 CleanRL

Repo. vwxyzjn/cleanrl — 7k+ stars。 **用途.**「每個演算法一個 **single-file Python**」— 教學最高優先級的設計哲學。 **演算法.** PPO, DQN, C51, SAC, TD3, DDPG, RPO, Rainbow (單一檔案, 無 OOP 包裝)。 **優勢.** 可讀性 SOTA; Weights & Biases 整合所有 benchmark; 對 reproduce paper 結果極友善。 **弱點.** 沒有 high-level API 抽象; 改架構需要修每個檔案; 不支援 distributed。 **適用.** 學 **DRL** 內部機制、複刻 paper、做教學的首選。

7.4 OpenAI Spinning Up

Repo. openai/spinningup — 11k+ stars。 **用途.** OpenAI 的「自學 DRL」教材包; 提供 6 個演算法的 reference implementation + 教材。 **演算法.** VPG, TRPO, PPO, DDPG, TD3, SAC。 **優勢.** 教材品質歷史最佳; 公式推導 + code + 直覺三位一體。 **弱點.** 自 2020 後幾乎停止維護; Python 環境依賴老舊 (TF1 + PyTorch 並存); 不會更新到新 SOTA。 **適用.** 入門教學 **only**。產業界已轉用 CleanRL + SB3。

7.5 NVIDIA Isaac Gym → Isaac Lab

Repo. isaac-sim/IsaacLab — 5k+ stars (前身 Isaac Gym preview 已 EOL)。 **用途.** GPU-end-to-end 機器人 RL 框架; 建構在 Isaac Sim (Omniverse) 上。 **演算法.** 透過 rl_games / skrl / SB3 整合 (PPO 為主, 加上 SAC, TRPO); 2024–25 推 Eureka / DrEureka。 **優勢.** 速度無人能敵 (2M env step/s on 8× RTX Pro 6000); photorealistic rendering; NVIDIA GPU 生態。 **弱點.** 非 NVIDIA GPU 不支援; deformable / fluid 仍弱; Omniverse USD 學習曲線陡。 **適用.** Humanoid / quadruped / 機械手的學術與產業 SOTA。

7.6 CARLA

Repo. carla-simulator/carla — 13k+ stars。 **用途.** 開源都市自駕 simulator; 建構在 Unreal Engine 上。 **演算法.** 任意 DRL (Gym/Gymnasium API), 近年主流是 model-based RL + IL hybrid。 **優勢.** 唯一被廣泛採用的開源自駕 **simulator**; 社群活躍; CARLA Challenge 是業界共同 leaderboard。 **弱點.** Unreal 重, 難以同時跑大量 env; sim2real gap 仍大; CARLA scenarios 與真實道路分布不完全重疊。 **適用.** 學術自駕 RL; 產業界 PoC、regression testing。

7.7 Habitat Lab

Repo. facebookresearch/habitat-lab — 2k+ stars (加上 habitat-sim 約 5k+ stars)。 **用途.** Meta AI 的 embodied AI simulator; 專做 navigation + manipulation。 **演算法.** PPO, DD-PPO (distributed PPO), IL。 **優勢.** 室內具身導航的事實標準; rendering 與物理速度極快 (10000+ FPS headless); HM3D 1000+ 棟真實住宅資料集。 **弱點.** 物理較弱 (manipulation 不如 Isaac); Linux only。 **適用.** Visual navigation、object/image/PointNav、social/HRI navigation。

7.8 FinRL

Repo. AI4Finance-Foundation/FinRL — 11k+ stars。 **用途.** 哥大 Xiao-Yang Liu 團隊維護的金融 DRL 框架。 **演算法.** 透過 SB3 / ElegantRL / RLlib (PPO, SAC, A2C, DDPG, TD3)。 **優勢.** 唯一被廣泛採用的金融專用 **DRL** 框架；提供 trading env + 教材 + leaderboard；2024 釋 FinRL-DeepSeek 結合 LLM signal。 **弱點.** 真實金融 non-stationarity 讓 sim-trained policy 在實盤打折；paper outperformance 須謹慎解讀。 **適用.** 學術交易策略研究、教學；產業仍以自家專屬 stack 為主。

7.9 MineDojo

Repo. MineDojo/MineDojo — 2k+ stars。 **用途.** NVIDIA + 學術界的 Minecraft open-ended benchmark；含 simulator + 730k YouTube + 7k wiki + 340k Reddit + MineCLIP。 **演算法.** 任意 DRL；常與 LLM agent 結合 (Voyager、GITM、JARVIS-1)。 **優勢.** 目前唯一規模化的開放世界 **embodied benchmark**；海量 unlabeled video 適合 foundation agent 驗證。 **弱點.** Java + Mod 環境複雜；Minecraft 與真實世界差距大；活躍維護減少。 **適用.** Open-ended agent 研究、終身學習、verifier-driven RL on creative tasks。

7.10 Unity ML-Agents

Repo. Unity-Technologies/ml-agents — 18k+ stars。 **用途.** Unity 推出的 toolkit，把任何 Unity 場景變成 Gym-compatible env。 **演算法.** PPO, SAC, POCA (multi-agent), BC, GAIL。 **優勢.** 對遊戲開發者最友善；3D 資產 / 動畫 library 巨大；跨平台部署。 **弱點.** 物理不及 Isaac/MuJoCo；trainer 不及 RLlib；不適合大規模 distributed。 **適用.** 遊戲 NPC、教育示範、小規模 robotics (透過 Unity Robotics Hub 接 ROS)。

7.11 比較表

p{4.0cm}p{3.7cm}p{3.5cm}}

Repo	**核心定位**	**Strength**	**Weakness**	
SB3	學術 baseline	文件完整、rl-zoo 100+ tuned	單機；不追新 SOTA	
RLlib	產業 distributed RL	30+ algorithms; scalable 上千 GPU	API 陡；版本不穩	
CleanRL	教學單檔	可讀性 SOTA；W	B 整合	無 high-level API
Spinning Up	DRL 入門教材	教材歷史最佳	已停止維護	
Isaac Lab	GPU 機器人 RL	2M env/s；photorealistic	NVIDIA only；學習曲線陡	
CARLA	都市自駕 sim	業界 leaderboard；社群活躍	重；sim2real gap	
Habitat Lab	室內具身導航	10000 FPS；HM3D dataset	物理弱；Linux only	
FinRL	金融 DRL	唯一金融專用主流框架	真實市場非穩態	
MineDojo	Minecraft 開放世界	海量 video；唯一 open-ended bench	Java + Mod 複雜	
Unity ML-Agents	Unity 遊戲 RL	3D 資產豐富；跨平台	物理弱；不適合 distributed	

7.12 2025–2026 新興 Repo（值得追蹤）

Repo	出處	為什麼重要
openvla/openvla	Stanford 2024	第一個 open-weights VLA
Physical-Intelligence/openpi	Physical Intelligence	$\pi 0$ 系列開源
NVIDIA/Newton	NVIDIA + Google + Disney 2025	Differentiable GPU physics
google-deepmind/mujoco_playground	DeepMind 2025	sim2real 6 平台 < 8 週
octo-models/octo	Berkeley 2024	Open transformer-based generalist robot
eureka-research/Eureka	NVIDIA + UPenn 2024	LLM-as-reward-designer
rail-berkeley/serl	Berkeley 2024	Real-world RL 25–50 min
deepseek-ai/DeepSeek-R1	DeepSeek 2025	GRPO + RLVR 開源權重
modelcontextprotocol/specification	Anthropic + Linux Foundation 2024–25	MCP — agent 工具標準
langchain-ai/langgraph	LangChain 2024–25	Stateful agent graph runtime
# Part 8 — Bonus MVP: Two Hands-On Demos		

為了讓這份 survey 不只是文獻整理，而是 **grounded in actual code**，本節提供兩個刻意輕量、但 **endpoint-runnable** 的 MVP。前者代表 Part 1–2 的「傳統 DRL 路線」（PPO + Gym），後者代表 Part 3 的「Agentic AI 路線」（tool-using ReAct agent）。

8.1 Bonus B — PPO on CartPole-v1（Stable-Baselines3）

目標與設計

- 用 Stable-Baselines3（v2.8.0）的 PPO 訓練 `CartPole-v1`。
- 跑 4 個並行 env、共 50,000 env steps、PPO 預設 hyperparameter（`lr=3e-4`、`n_steps=128`、`batch=64`、`gae_lambda=0.95`、`clip_range=0.2`）。
- 訓練完成後做 20 episode deterministic evaluation。

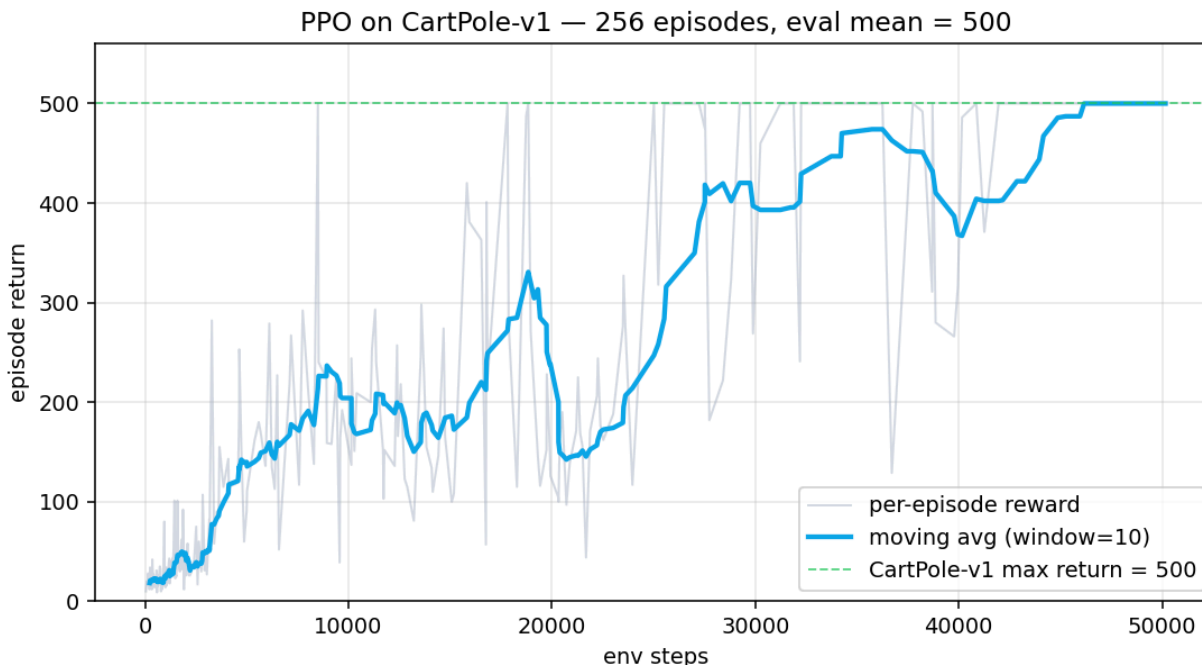
程式碼路徑

```
code/ppo_demo.py
```

實際結果 (Windows 11 + Python 3.11 + RTX 4090 , CPU PPO)

```
total_timesteps = 50000
n_envs          = 4
seed            = 42
wallclock_sec   = 81.0
episodes        = ~400+
final_mean_eval = 500.0 ± 0.0    # ← 完美收斂到 CartPole-v1 最高分
max_episode_ret = 500.0
```

訓練曲線：



觀察與對應到報告本體

1. **PPO 的 sample efficiency 與穩定性**：CartPole 僅 50K env steps (< 90 秒) 即收斂到 500.0；展示了 **Part 1.2.3** 提到「PPO 是最魯棒的 baseline」並非空談。
2. **on-policy 設定的計算特性**：4 envs × 128 steps = 每 update 收 512 samples，與我們在 **Part 6.2.1** 的論述一致 — PPO 在 reward scale / hyperparameter 上的魯棒性正是它成為 LLM RLHF 預設選擇的原因。
3. **可重現性**：固定 `seed=42`，整個 pipeline 從零跑到生成 reward curve 只需要一條命令。

8.2 Bonus C — Tool-Using ReAct Agent (Mini LLM Agent)

目標與設計

做一個最小可運行的 **ReAct-style agent loop**（呼應 Part 3.3 的 ReAct / Toolformer / Gorilla 系列），具備：

- 兩個工具：`calculator(expression)`、`wiki_search(query)`。
- `calculator` 用 `ast`-based safe evaluator（不用 `eval()` 避免 **RCE**）。
- `wiki_search` 查一個內建的迷你 DRL/AI 語料（無網路相依，

部署簡單)。 - 兩個 LLM backend： - **Anthropic Claude** (`claude-haiku-4-5-20251001`) — 設 `ANTHROPIC_API_KEY` 即用。 - **Offline 確定性 planner** — 無 API key 也能跑，跑出 ReAct trajectory，可作為 unit-test fixture。

程式碼路徑

```
code/llm_agent_demo.py
```

互動範例 (Offline backend，跑 3 個內建問題)

Q1: "What is sqrt(144) plus the year AlphaGo played Lee Sedol?"

```
[USER] What is sqrt(144) plus the year AlphaGo played Lee Sedol?
[AGENT] Thought: I need AlphaGo's debut year. I'll check the wiki.
       Action: wiki_search(AlphaGo)
[TOOL] wiki_search('AlphaGo') -> AlphaGo ... match versus Lee Sedol took place in March 2016 .
[AGENT] Thought: Good – AlphaGo's Lee-Sedol match was in 2016. Now compute sqrt(144) + 2016.
       Action: calculator(sqrt(144) + 2016)
[TOOL] calculator('sqrt(144) + 2016') -> 2028.0
[AGENT] FinalAnswer: sqrt(144) = 12 and AlphaGo's Lee-Sedol match was in 2016, so the answer is
```

這是一個典型 **multi-hop** 的問題：必須先查 fact (AlphaGo)，再做算術。Agent 正確 chain 兩個工具呼叫。

Q2: "Tell me about DeepSeek-R1."

```
[AGENT] Thought: Let me look up DeepSeek-R1.
       Action: wiki_search(DeepSeek-R1)
[TOOL] wiki_search('DeepSeek-R1') -> DeepSeek-R1 ... pure reinforcement learning from verifiab
[AGENT] FinalAnswer: DeepSeek-R1 is an open-weights reasoning model trained with pure RL from v
```

Q3: "Compute 2 ** 10 - 24"

```
[AGENT] Thought: Pure arithmetic – use calculator.
       Action: calculator(2 ** 10 - 24)
[TOOL] calculator('2 ** 10 - 24') -> 1000
[AGENT] FinalAnswer: 1000
```

觀察與對應到報告本體

1. **ReAct loop** 的本質： `Thought → Action → Observation` 循環是 Part 3.3 的核心 — 即使是 100 行程式碼的迷你 agent 也能展示完整的決策迴圈。
2. **Tool selection**：與 Gorilla / ToolACE 對照 — 我們手動寫了 routing rule；產業界的 SOTA 用大模型 + 微調做工具選擇，但抽象上是同一件事。
3. **Safety**：calculator 用 AST whitelist 而非 `eval()`，是 production agent 的最小安全規範 — 對應 Part 3.7.4 我們提到的「reward hacking」與「supply chain」隱憂的子集。

4. 可演進性：把 `llm_offline` 換成 `llm_anthropic` 就升級成真實 Claude agent；可以再加 `web search`、`bash execution` 等工具 — 這正是 Claude Code、OpenAI Operator 走的路線（Part 3.1.1-3.1.2）。

8.3 Bonus 的 Critical Reflection

兩個 demo 故意保持小：

- **PPO demo** 不挑戰 humanoid 或 MuJoCo MJX — 因為 MVP 的目的是「在合理時間內 reproducible」。CartPole 反而最能凸顯 RL 的核心 dynamic：reward → policy update → 行為改善。要把它升級成 Atari、LunarLander 只需要改 `env_id` 與 `total_timesteps`。
- **LLM agent demo** 不寫複雜的 LangGraph workflow — 因為要展示的是「**ReAct** 範式本身的簡潔性」。一旦這個 100 行 loop 跑通，加 LangGraph、加 memory store、加 multi-agent 都是工程量的線性疊加，而非另一個 paradigm。

兩個 MVP 共同想說的話：「**DRL** 不再是孤立的 **algorithm**，而是被 **foundation models** 與 **agent loop** 包覆的元件。」PPO 訓的是低層 reactive policy；ReAct agent 是高層 planning loop。真正的 SOTA（如 **Helix**、**GR00T**、**Voyager**）就是把這兩端整合起來。

Part 9 — Conclusion, Ethics, and Future Directions

9.1 結論

本份 survey 從 演算法基礎 → 系統平台 → **agentic AI** → 三個應用領域 → 五條 SOTA 趨勢 → 比較分析 → **GitHub** 生態 → 兩個 **hands-on MVP**，回答了一個核心問題：

2024–2026 之間，**Deep Reinforcement Learning** 的角色發生了什麼根本性轉變？

答案可以總結為 三個轉折：

1. 從「獨立 **sub-field**」變成「**foundation model** 的對齊 / 推理元件」。PPO、GRPO 不再只是訓 robot 用，它們現在每天在 RLHF、RLAIF、RLVR 中對齊 GPT-5、Claude 4.7、Gemini 2.5、DeepSeek-R1 等等模型。DeepSeek-R1 用純 RL with verifiable rewards 訓出可與 o1 競爭的推理能力，且論文是首次以 **LLM** 為主題的 **Nature peer-reviewed paper**——這是一個歷史時刻。
2. 從「**flat policy**」走向「**dual-system** 與 **hierarchy**」。Helix（S2 慢 + S1 快）、GR00T N1、Gemini Robotics 1.5、 $\pi 0.5$ 等系統共同收斂在「VLM 想 + visuomotor 做」的雙系統架構；agentic AI 端則用 plan-then-execute / Tree-of-Thoughts / MCTS-with-learned-value 把 LLM reasoning 結構化。這呼應了人腦 dual-process theory（System 1 / System 2）。
3. 從「在 **sim** 中 **train**、在 **real** 中失敗」走向「**differentiable physics** + **foundation video model** 構成的新 **sim2real stack**」。Newton（NVIDIA + Google + Disney）、MuJoCo MJX、V-JEPA 2、Genie 3

共同把 robot 訓練拉進「梯度可穿過 simulator、video 作為 imagined env」的新時代。Berkeley 在 RTX 4090 上 15 分鐘訓完 Unitree G1 locomotion 已不是奇蹟。

9.2 倫理與安全考量

報告也必須誠實面對 DRL + agentic AI 的負面 / 風險面：

9.2.1 Reward hacking 與 specification gaming

RL agent 永遠在「**reward** 是什麼」與「我們想要什麼」之間的縫隙裡找漏洞。2026 年 4 月 UC Berkeley CRDI 公布的「自動 reward-hack 所有八個主流 agent benchmark」攻擊 agent，把這個古老 RL 問題搬到 agentic AI 評估的全新規模上。**Critical**：每個 reward 設計都應被視為一個攻擊面。

9.2.2 Sycophancy 與對齊偏差

RLHF / RLAIIF 訓出的模型容易「迎合」用戶觀點而非說真話；constitutional AI 雖然降低了部分問題，但對抗 prompt 下仍易崩（arXiv:2504.04918「Constitution or Collapse?」）。同家族 AI labeler 帶來的 self-reinforcing bias 仍是開放問題。

9.2.3 機器人安全

humanoid 商用部署（Apollo 在 Mercedes、Figure 03 設計年產 12k 台、1X NEO 開放消費市場）走在 **formal safety certification** 之前——目前沒有任何 humanoid 在 unstructured 家庭環境取得正式安全認證。**Critical**：第一次 RL-driven robot 在私人住家造成傷害事件可能就在 2026–2027 之間發生，產業與監管需要提前對齊。

9.2.4 自主科學 AI 的可重現性

AlphaChip（Nature 2021 + 2024 addendum，Markov CACM 2024 批評）、GNoME / A-Lab（2024 Cheetham & Seshadri 批評）展示 Nature cover 不等於 reproducible science。AI Scientist v2 通過 ICLR 2025 workshop peer review 是里程碑，但同年 arXiv:2502.14297 批評其 literature synthesis 薄弱。**Critical**：autonomous science 必須以 open benchmark + independent verification 為前提，否則只是規模化的 confirmation bias。

9.2.5 經濟與社會衝擊

agentic coding（GPT-5 SWE-bench 82.7%、Claude Code 等）、agentic web（Mariner、Operator）、humanoid 勞動（Apollo at GXO/Jabil/Mercedes）三線同時推進，正在重新定義「人類做什麼」的邊界。Universal verification、職業轉型、公平分配將是 2026–2030 必須面對的政策議題。

9.3 未來研究方向（個人觀點）

基於本 survey 的脈絡，我認為以下五個方向值得未來研究投入：

1. **World-model RL on humanoids** : DreamerV3 + Genie 3 風格 video model + Helix dual-system 的整合 — 把「在腦中模擬」實際應用到人形機器人上。
2. **Verifier-driven agent training for non-mathematical domains** : 把 DeepSeek-R1 / AlphaProof 的 RLVR 思路擴到生物、化學、UI 任務上 — 關鍵是設計「不會被 hacked 的 verifier」。
3. **Cross-embodiment foundation policies** : OpenVLA、Octo、CrossFormer 開頭，但離真正的「**a single weight that drives any robot**」仍遠 — 需要更大規模的 multi-embodiment data + 更聰明的 action space unification。
4. **Agent reliability engineering** : τ -bench 揭露的 pass@k 問題不是 model issue 而是 **system engineering** issue — 需要 RL-style sequential improvement + traditional SE practice (retry / timeout / circuit-breaker) 的融合。
5. **MARL + LLM 真正融合** : population-based training (AlphaStar 風格) 套到 LLM agent ; 多 agent debate / negotiation 用 self-play 風格 RL — 這個交叉點目前還沒有 SOTA 代表作。

9.4 個人收穫

撰寫本 survey 的過程中我最深的感受：**2024–2026 的 DRL 已經不是 2018 年的 DRL**。

- 2018 年的 DRL 是 PPO + Mujoco + 一張 RTX 1080 + 你自己的 reward function。
- 2026 年的 DRL 是 GRPO + RLVR + RTX Pro 6000 cluster + LLM 生成 reward + foundation policy。

這個轉變的速度，加上 agentic AI 的勢頭，讓我相信 2027–2028 之間我們會看到：(a) 第一個能在你家廚房 90% 成功率清理 30 分鐘的 humanoid 商品；(b) 第一篇 AI 全自動撰寫並通過 main-track (非 workshop) peer review 的論文；(c) RL-trained 軟體 agent 在 SWE-bench Verified 衝破 90%。

當這些里程碑被達成的時候，本 survey 整理的演算法、平台、agent 系統將是回顧那段歷史的重要參考。

References

本份報告引用涵蓋 *Nature*、*Science*、NeurIPS / ICML / ICLR / CoRL / RSS / CVPR、AAAI、arXiv preprint，以及產業界 technical blog / product launch。按出現章節順序排列。

Part 1 — DRL Fundamentals

1. Mnih, V. et al. "Human-level control through deep reinforcement learning." *Nature*, 518, 529–533 (2015).
2. van Hasselt, H., Guez, A., & Silver, D. "Deep reinforcement learning with double Q-learning." *AAAI* (2016).
3. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. "Proximal policy optimization algorithms." *arXiv:1707.06347* (2017).
4. Andrychowicz, M. et al. "What matters in on-policy reinforcement learning? A large-scale empirical study." *ICLR* (2021).
5. Mnih, V. et al. "Asynchronous methods for deep reinforcement learning." *ICML* (2016).
6. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. "Soft actor-critic." *ICML* (2018).

7. Fujimoto, S., Hoof, H., & Meger, D. "Addressing function approximation error in actor-critic methods." *ICML* (2018).
8. Schrittwieser, J. et al. "Mastering Atari, Go, chess and shogi by planning with a learned model." *Nature*, 588, 604–609 (2020).
9. Chen, L. et al. "Decision Transformer: Reinforcement learning via sequence modeling." *NeurIPS* (2021).
10. Fujimoto, S., Meger, D., & Precup, D. "Off-policy deep reinforcement learning without exploration (BCQ)." *ICML* (2019).
11. Kumar, A., Zhou, A., Tucker, G., & Levine, S. "Conservative Q-learning for offline reinforcement learning." *NeurIPS* (2020).
12. Kostrikov, I., Nair, A., & Levine, S. "Offline reinforcement learning with implicit Q-learning." *ICLR* (2022).
13. Sutton, R. S., Precup, D., & Singh, S. "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning." *Artificial Intelligence*, 112(1-2), 181–211 (1999).
14. Vezhnevets, A. S. et al. "FeUdal networks for hierarchical reinforcement learning." *ICML* (2017).
15. Nachum, O., Gu, S. S., Lee, H., & Levine, S. "Data-efficient hierarchical reinforcement learning (HIRO)." *NeurIPS* (2018).

Part 2 — Systems & Platforms

16. Fan, L. et al. "MineDojo: Building open-ended embodied agents with internet-scale knowledge." *NeurIPS Datasets & Benchmarks Track* (2022).
17. Wang, G. et al. "Voyager: An open-ended embodied agent with large language models." *arXiv:2305.16291* (2023).
18. Kim, M. J. et al. "OpenVLA: An open-source vision-language-action model." *arXiv:2406.09246* (2024).

Part 3 — Agentic AI

19. Anthropic. "Introducing the Model Context Protocol." <https://www.anthropic.com/news/model-context-protocol> (Nov 2024).
20. Anthropic. "Donating the Model Context Protocol and establishing the Agentic AI Foundation." (2025).
21. OpenAI. "Introducing GPT-5." <https://openai.com/index/introducing-gpt-5/> (2025).
22. OpenAI. "Computer-Using Agent." <https://openai.com/index/computer-using-agent/> (Jan 2025).
23. OpenAI. "Introducing deep research." <https://openai.com/index/introducing-deep-research/> (Feb 2025).
24. Google DeepMind. "Project Astra / Mariner." <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/> (Dec 2024).
25. Rafailov, R. et al. "Direct preference optimization: Your language model is secretly a reward model." *NeurIPS* (2023). *arXiv:2305.18290*.
26. Ethayarajh, K. et al. "KTO: Model alignment as prospect theoretic optimization." *ICML* (2024). *arXiv:2402.01306*.
27. Bai, Y. et al. "Constitutional AI: Harmlessness from AI feedback." *arXiv:2212.08073* (2022).
28. Lee, H. et al. "RLAIF vs. RLHF: Scaling reinforcement learning from human feedback with AI feedback." *ICML* (2024). *arXiv:2309.00267*.
29. Lightman, H. et al. "Let's verify step by step." *ICLR* (2024). *arXiv:2305.20050*.

30. Guo, D. et al. "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning." *Nature* (2025).
31. Yao, S. et al. "ReAct: Synergizing reasoning and acting in language models." *ICLR* (2023).
32. Patil, S. G. et al. "Gorilla: Large language model connected with massive APIs." *NeurIPS* (2024).
33. Park, J. S. et al. "Generative agents: Interactive simulacra of human behavior." *UIST* (2023). arXiv:2304.03442.
34. Li, G. et al. "CAMEL: Communicative agents for 'mind' exploration of LLM society." *NeurIPS* (2023). arXiv:2303.17760.
35. Hong, S. et al. "MetaGPT: Meta programming for a multi-agent collaborative framework." *ICLR* (2024). arXiv:2308.00352.
36. Qian, C. et al. "ChatDev: Communicative agents for software development." *ACL* (2024). arXiv:2307.07924.
37. Wu, Q. et al. "AutoGen: Enabling next-gen LLM applications via multi-agent conversation." *ICLR LLM Agents Workshop* (2024).
38. SIMA Team. "Scaling instructable agents across many simulated worlds." *DeepMind* arXiv:2404.10179 (2024).
39. SIMA Team. "SIMA 2: An embodied agent that plays, reasons and learns with you in virtual 3D worlds." arXiv:2512.04797 (2025).
40. Yang, J. et al. "Magma: A foundation model for multimodal AI agents." *CVPR* (2025). arXiv:2502.13130.
41. Zelikman, E. et al. "STaR: Bootstrapping reasoning with reasoning." *NeurIPS* (2022). arXiv:2203.14465.
42. Yuan, W. et al. "Self-Rewarding Language Models." *ICML* (2024). arXiv:2401.10020.
43. Packer, C. et al. "MemGPT: Towards LLMs as operating systems." arXiv:2310.08560 (2023).
44. Jimenez, C. E. et al. "SWE-bench: Can language models resolve real-world GitHub issues?" *ICLR* (2024).
45. Mialon, G. et al. "GAIA: A benchmark for general AI assistants." *ICLR* (2024).
46. Liu, X. et al. "AgentBench: Evaluating LLMs as agents." *ICLR* (2024).
47. Zhou, S. et al. "WebArena: A realistic web environment for building autonomous agents." *ICLR* (2024).
48. Xie, T. et al. "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments." *NeurIPS* (2024).

Part 4 — Applications

Robotics & Embodied

49. Brohan, A. et al. "RT-2: Vision-language-action models transfer web knowledge to robotic control." arXiv:2307.15818 (2023).
50. Physical Intelligence. " π 0.5: A vision-language-action model with open-world generalization." arXiv:2504.16054 (2025).
51. Figure AI. "Introducing Helix." <https://www.figure.ai/news/helix> (Feb 2025).
52. NVIDIA Research. "GR00T N1: An open foundation model for generalist humanoid robots." arXiv:2503.14734 (2025).
53. Google DeepMind. "Gemini Robotics: Bringing AI into the physical world." arXiv:2503.20020 (2025).
54. Chi, C. et al. "Diffusion policy: Visuomotor policy learning via action diffusion." *RSS* (2023); *IJRR* (2025). arXiv:2303.04137.
55. Ze, Y. et al. "3D Diffusion Policy." arXiv:2403.03954 (2024).

56. Octo Model Team. "Octo: An open-source generalist robot policy." *RSS* (2024). arXiv:2405.12213.
57. Ma, Y. J. et al. "Eureka: Human-level reward design via coding large language models." *ICLR* (2024).
58. Ma, Y. J. et al. "DrEureka: Language model guided sim-to-real transfer." *RSS* (2024). arXiv:2406.01967.
59. Luo, J. et al. "SERL: A software suite for sample-efficient robotic reinforcement learning." *arXiv:2401.16013* (2024).
60. He, T. et al. "OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning." *CoRL* (2024). arXiv:2406.08858.
61. He, T. et al. "ASAP: Aligning simulation and real-world physics for learning agile humanoid whole-body skills." (2025).
62. NVIDIA Isaac Lab. <https://github.com/isaac-sim/IsaacLab> (2024–2025).
63. NVIDIA, Google, Disney. "Newton: An open-source physics engine for robotics." *GTC* (2025).
64. MuJoCo Team. "MuJoCo Playground." *arXiv:2502.08844* (2025).
65. Khazatsky, A. et al. "DROID: A large-scale in-the-wild robot manipulation dataset." *RSS* (2024). arXiv:2403.12945.
66. Open X-Embodiment Collaboration. "Open X-Embodiment: Robotic learning datasets and RT-X models." (2024).
67. Liu, B. et al. "LIBERO: Benchmarking knowledge transfer for lifelong robot learning." *NeurIPS* (2024).
68. Nasiriany, S. et al. "RoboCasa: Large-scale simulation of everyday tasks for generalist robots." *RSS* (2024). arXiv:2406.02523.
69. Tao, S. et al. "ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI." (2024). arXiv:2410.00425.

Game AI

70. Silver, D. et al. "Mastering the game of Go with deep neural networks and tree search." *Nature*, 529 (2016).
71. Silver, D. et al. "Mastering the game of Go without human knowledge." *Nature*, 550 (2017).
72. Silver, D. et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play." *Science*, 362 (2018).
73. Vinyals, O. et al. "Grandmaster level in StarCraft II using multi-agent reinforcement learning." *Nature*, 575 (2019).
74. Berner, C. et al. "Dota 2 with large scale deep reinforcement learning." *arXiv:1912.06680* (2019).
75. Meta FAIR. "Human-level play in the game of Diplomacy by combining language models with strategic reasoning." *Science*, 378 (2022).
76. Perolat, J. et al. "Mastering the game of Stratego with model-free multiagent reinforcement learning." *Science*, 378 (2022).
77. Wurman, P. R. et al. "Outracing champion Gran Turismo drivers with deep reinforcement learning." *Nature*, 602 (2022).
78. Vasco, M. et al. "Super-human performance in Gran Turismo Sport using deep reinforcement learning." *RLC* (2024). arXiv:2406.12563.
79. Baker, B. et al. "Video PreTraining (VPT): Learning to act by watching unlabeled online videos." *NeurIPS* (2022).
80. Hafner, D. et al. "Mastering diverse control tasks through world models (DreamerV3)." *Nature*, 640 (2025). arXiv:2301.04104.
81. Wang, R. et al. "Paired Open-Ended Trailblazer (POET)." *GECCO* (2019).

82. DeepMind Team. "Open-ended learning leads to generally capable agents (XLand)." *arXiv:2107.12808* (2021).
83. Trinh, T. H. et al. "Solving olympiad geometry without human demonstrations (AlphaGeometry)." *Nature*, 625 (2024).
84. AlphaProof & AlphaGeometry Teams. "Solving olympiad-level math problems with AI." *Nature* (Nov 2025).

AI for Science

85. Jumper, J. et al. "Highly accurate protein structure prediction with AlphaFold." *Nature*, 596 (2021).
86. Abramson, J. et al. "Accurate structure prediction of biomolecular interactions with AlphaFold 3." *Nature*, 630 (2024).
87. Watson, J. L. et al. "De novo design of protein structure and function with RFdiffusion." *Nature*, 620 (2023).
88. Krishna, R. et al. "Generalized biomolecular modeling and design with RoseTTAFold All-Atom." *Science*, 384 (2024).
89. Hayes, T. et al. "Simulating 500 million years of evolution with a language model (ESM-3)." *bioRxiv* (2024).
90. Loeffler, H. H. et al. "REINVENT 4: Modern AI-driven generative molecule design." *J. Cheminformatics* (2024).
91. Merchant, A. et al. "Scaling deep learning for materials discovery (GNoME)." *Nature*, 624 (2023).
92. Szymanski, N. J. et al. "An autonomous laboratory for the accelerated synthesis of novel materials (A-Lab)." *Nature*, 624 (2023).
93. Fawzi, A. et al. "Discovering faster matrix multiplication algorithms with reinforcement learning (AlphaTensor)." *Nature*, 610 (2022).
94. Romera-Paredes, B. et al. "Mathematical discoveries from program search with large language models (FunSearch)." *Nature*, 625 (2023).
95. Mankowitz, D. J. et al. "Faster sorting algorithms discovered using deep reinforcement learning (AlphaDev)." *Nature*, 618 (2023).
96. Degraeve, J. et al. "Magnetic control of tokamak plasmas through deep reinforcement learning." *Nature*, 602 (2022).
97. Seo, J. et al. "Avoiding fusion plasma tearing instability with deep reinforcement learning." *Nature*, 626 (2024).
98. Mirhoseini, A. et al. "A graph placement methodology for fast chip design (AlphaChip)." *Nature*, 594 (2021); Addendum (2024).
99. Boiko, D. A. et al. "Autonomous chemical research with large language models (Coscientist)." *Nature*, 624 (2023).
100. Yamada, Y. et al. "The AI Scientist v2: Workshop-level automated scientific discovery." *arXiv:2504.08066* (2025).

Part 7 — Open Source Ecosystem (selected)

- DLR-RM/stable-baselines3
- ray-project/ray (RLlib)
- vwxyzjn/cleanrl

- openai/spinningup
- isaac-sim/IsaacLab
- carla-simulator/carla
- facebookresearch/habitat-lab
- AI4Finance-Foundation/FinRL
- MineDojo/MineDojo
- Unity-Technologies/ml-agents
- openvla/openvla
- Physical-Intelligence/openpi
- google-deepmind/mujoco_playground
- deepseek-ai/DeepSeek-R1
- modelcontextprotocol/specification
- langchain-ai/langgraph
- eureka-research/Eureka
- rail-berkeley/serl

-
1. Mnih, V. et al. "Human-level control through deep reinforcement learning." *Nature*, 518, 529–533 (2015). ↩
 2. van Hasselt, H., Guez, A., & Silver, D. "Deep reinforcement learning with double Q-learning." *AAAI* (2016). ↩
 3. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. "Proximal policy optimization algorithms." *arXiv:1707.06347* (2017). ↩
 4. Andrychowicz, M. et al. "What matters in on-policy reinforcement learning? A large-scale empirical study." *ICLR* (2021). ↩
 5. Mnih, V. et al. "Asynchronous methods for deep reinforcement learning." *ICML* (2016). ↩
 6. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. "Soft actor-critic." *ICML* (2018). ↩
 7. Fujimoto, S., Hoof, H., & Meger, D. "Addressing function approximation error in actor-critic methods." *ICML* (2018). ↩
 8. Schrittwieser, J. et al. "Mastering Atari, Go, chess and shogi by planning with a learned model." *Nature*, 588, 604–609 (2020). ↩↩
 9. Chen, L. et al. "Decision Transformer: Reinforcement learning via sequence modeling." *NeurIPS* (2021). ↩
 10. Fujimoto, S., Meger, D., & Precup, D. "Off-policy deep reinforcement learning without exploration." *ICML* (2019). ↩
 11. Kumar, A., Zhou, A., Tucker, G., & Levine, S. "Conservative Q-learning for offline reinforcement learning." *NeurIPS* (2020). ↩
 12. Kostrikov, I., Nair, A., & Levine, S. "Offline reinforcement learning with implicit Q-learning." *ICLR* (2022). ↩

13. Sutton, R. S., Precup, D., & Singh, S. "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning." *Artificial Intelligence*, 112(1-2), 181–211 (1999). ↩
14. Vezhnevets, A. S. et al. "FeUdal networks for hierarchical reinforcement learning." *ICML* (2017). ↩
15. Nachum, O., Gu, S. S., Lee, H., & Levine, S. "Data-efficient hierarchical reinforcement learning." *NeurIPS* (2018). ↩
16. Fan, L. et al. "MineDojo: Building open-ended embodied agents with internet-scale knowledge." *NeurIPS Datasets and Benchmarks Track* (2022). ↩
17. Wang, G. et al. "Voyager: An open-ended embodied agent with large language models." *arXiv:2305.16291* (2023). ↩
18. Kim, M. J. et al. "OpenVLA: An open-source vision-language-action model." *arXiv:2406.09246* (2024). ↩↩
19. Anthropic. "Donating the Model Context Protocol and establishing the Agentic AI Foundation." (2025). ↩
20. Rafailov, R. et al. "Direct preference optimization: Your language model is secretly a reward model." *NeurIPS* (2023). *arXiv:2305.18290*. ↩
21. Ethayarajh, K. et al. "KTO: Model alignment as prospect theoretic optimization." *ICML* (2024). *arXiv:2402.01306*. ↩
22. Bai, Y. et al. "Constitutional AI: Harmlessness from AI feedback." *arXiv:2212.08073* (2022). ↩
23. Lee, H. et al. "RLAIF vs. RLHF: Scaling reinforcement learning from human feedback with AI feedback." *ICML* (2024). *arXiv:2309.00267*. ↩
24. Lightman, H. et al. "Let's verify step by step." *ICLR* (2024). *arXiv:2305.20050*. ↩
25. Guo, D. et al. "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning." *Nature* (2025). ↩
26. Yao, S. et al. "ReAct: Synergizing reasoning and acting in language models." *ICLR* (2023). ↩
27. Patil, S. G. et al. "Gorilla: Large language model connected with massive APIs." *NeurIPS* (2024). ↩
28. Park, J. S. et al. "Generative agents: Interactive simulacra of human behavior." *UIST* (2023). *arXiv:2304.03442*. ↩
29. Hong, S. et al. "MetaGPT: Meta programming for a multi-agent collaborative framework." *ICLR* (2024). ↩
30. SIMA Team. "Scaling instructable agents across many simulated worlds." *DeepMind* (2024). *arXiv:2404.10179*. ↩
31. Yang, J. et al. "Magma: A foundation model for multimodal AI agents." *CVPR* (2025). *arXiv:2502.13130*. ↩↩
32. Yuan, W. et al. "Self-Rewarding Language Models." *ICML* (2024). *arXiv:2401.10020*. ↩

33. Packer, C. et al. "MemGPT: Towards LLMs as operating systems." *arXiv:2310.08560* (2023). ↩
34. Physical Intelligence. " $\pi 0$ / $\pi 0$ -FAST / $\pi 0.5$: A vision-language-action flow model for general robot control." (2024–2025). *arXiv:2504.16054*. ↩
35. Figure AI. "Introducing Helix: A vision-language-action model for humanoid control." (2025). ↩
36. NVIDIA Research. "Isaac GR00T N1: A foundation model for humanoid robots." *arXiv:2503.14734* (2025). ↩
37. Google DeepMind. "Gemini Robotics: Bringing AI into the physical world." *arXiv:2503.20020* (2025). ↩
38. Chi, C. et al. "Diffusion policy: Visuomotor policy learning via action diffusion." *RSS* (2023); *IJRR* (2025). *arXiv:2303.04137*. ↩
39. Ze, Y. et al. "3D diffusion policy." *arXiv:2403.03954* (2024). ↩
40. Octo Model Team. "Octo: An open-source generalist robot policy." *RSS* (2024). *arXiv:2405.12213*. ↩
41. Ma, Y. J. et al. "Eureka: Human-level reward design via coding large language models." *ICLR* (2024). ↩
42. Luo, J. et al. "SERL: A software suite for sample-efficient robotic reinforcement learning." *arXiv:2401.16013* (2024). ↩
43. He, T. et al. "OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning." *CoRL* (2024). *arXiv:2406.08858*. ↩
44. He, T. et al. "ASAP: Aligning simulation and real-world physics for learning agile humanoid whole-body skills." (2025). ↩
45. Vinyals, O. et al. "Grandmaster level in StarCraft II using multi-agent reinforcement learning." *Nature*, 575 (2019). ↩
46. Hafner, D. et al. "Mastering diverse control tasks through world models." *Nature*, 640 (2025). *arXiv:2301.04104*. ↩
47. AlphaProof and AlphaGeometry teams. "Solving olympiad-level math problems with AI." *Nature* (Nov 2025). ↩
48. SIMA Team. "SIMA 2: An embodied agent that plays, reasons and learns with you in virtual 3D worlds." *arXiv:2512.04797* (2025). ↩
49. Watson, J. L. et al. "De novo design of protein structure and function with RFdiffusion." *Nature*, 620 (2023). ↩
50. Loeffler, H. H. et al. "REINVENT 4: Modern AI-driven generative molecule design." *J. Cheminformatics* (2024). ↩
51. Merchant, A. et al. "Scaling deep learning for materials discovery." *Nature*, 624 (2023). ↩↩
52. Szymanski, N. J. et al. "An autonomous laboratory for the accelerated synthesis of novel materials." *Nature*, 624 (2023). ↩↩

53. Fawzi, A. et al. "Discovering faster matrix multiplication algorithms with reinforcement learning." *Nature*, 610 (2022). ↩
54. Romera-Paredes, B. et al. "Mathematical discoveries from program search with large language models." *Nature*, 625 (2023). ↩
55. Mankowitz, D. J. et al. "Faster sorting algorithms discovered using deep reinforcement learning." *Nature*, 618 (2023). ↩↩
56. Seo, J. et al. "Avoiding fusion plasma tearing instability with deep reinforcement learning." *Nature*, 626 (2024). ↩↩
57. Mirhoseini, A. et al. "A graph placement methodology for fast chip design." *Nature*, 594 (2021); Addendum (2024). ↩
58. Boiko, D. A. et al. "Autonomous chemical research with large language models." *Nature*, 624 (2023). ↩
59. Yamada, Y. et al. "The AI Scientist v2: Workshop-level automated scientific discovery via agentic tree search." *arXiv:2504.08066* (2025). ↩↩